Bahauddin Zakariya University, Multan, Pakistan

Department of Information Technology

Domavi: Smart Micro-Rental and Roommate Matching Platform

A Software Requirements Specification (SRS) and Software Design Document (SDD) Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Science in Information Technology (BS-IT)

Submitted By

Qamar Shahzad (BSIT-21-41)
Zain Irshad (BSIT-21-45)

Supervisor

Sir Toseefullah

Session: 2021 – 2025

Dedication

I dedicate this work to my parents, whose unwavering support, sacrifices, and prayers have been the foundation of my success. Their encouragement, patience, and belief in my abilities have guided me throughout this journey.

I also dedicate this to my teachers and mentors who inspired me to pursue knowledge with dedication and integrity.

Acknowledgement

All praise and gratitude are due for the successful completion of this project. The authors would like to express their sincere appreciation to their supervisor for continuous guidance, valuable suggestions, and constructive feedback throughout the development of this thesis. The supervision and mentorship provided significant support in completing this work successfully.

Special thanks are extended to the faculty members of the Department of Information Technology, Bahauddin Zakariya University, Multan, for their academic support and encouragement during the degree program.

The authors are also deeply grateful to their parents and families for their unwavering support, motivation, and prayers, which have been a constant source of strength.

Finally, appreciation is extended to everyone who directly or indirectly contributed to the successful completion of this project.

Certificate of Approval

This is to certify that the Final Year Project titled **"Domavi: Smart Micro-Rental and Roommate Matching Platform"** has been developed by **Qamar Shahzad (BSIT-21-41)** and **Zain Irshad (BSIT-21-45)** under the supervision of **Sir Toseefullah** and that in his opinion, the project is adequate in scope and quality for the award of the degree of Bachelor of Science in Information Technology.

_____

Supervisor
Sir Toseefullah

_____

External Examiner
Prof. Dr. Maruf Pasha

_____

Head of Department
Sir Mohsin Naqi
Department of Information Technology

Plagiarism and AI Content Certificate

Date: _____

It is hereby certified that the similarity index (as generated by the Turnitin Originality Report) of the **Final Year Project Report (SRS / SDD)** submitted by:

- **Qamar Shahzad (BSIT-21-41)**
- **Zain Irshad (BSIT-21-45)**

titled:

"Domavi: Smart Micro-Rental and Roommate Matching Platform"

is **[XX%]** which is within the permissible limits as defined by the Higher Education Commission (HEC) rules and regulations. The similarity index report is attached herewith.

Furthermore, the AI-generated content detection index (if applicable as per institutional policy) is reported as **[XX%]**. The report has been reviewed, and the content is found to comply with university guidelines regarding ethical use of Artificial Intelligence tools.

The report is recommended for submission.

Supervisor Name: _____

Designation: _____

Signature: _____

Department: Department of Information Technology

University: Bahauddin Zakariya University, Multan

Executive Summary

Affordable housing remains one of the most pressing challenges faced by students and young professionals across Pakistan. The traditional rental process is dominated by informal brokers, unverified listings, and a complete absence of any structured method for finding a compatible roommate. As a result, tenants frequently encounter inflated brokerage commissions, fraudulent advertisements, security risks, and stressful living arrangements caused by mismatched roommates. Existing online platforms are oriented toward full-house rentals and provide no support for single rooms, bed spaces, or shared accommodation located near universities and workplaces.

This project, titled **Domavi: Smart Micro-Rental and Roommate Matching Platform**, proposes a unified, web-based ecosystem that directly addresses these gaps. Domavi combines a dedicated micro-rental marketplace for single rooms, bed spaces, and hostels with an intelligent, algorithm-driven roommate matching engine that scores potential roommates on the basis of lifestyle, preferences, budget, location, and daily schedule compatibility. To establish trust, the platform enforces a multi-layer verification system covering email and phone confirmation, administrator-reviewed CNIC verification, and property ownership validation. A commute-based search powered by the Google Maps API enables users to discover accommodation within walking distance of their university or office, while secure in-app messaging, structured booking requests, online payment recording, and auto-generated digital rental agreements automate the entire rental lifecycle.

Domavi is implemented using the MERN technology stack, comprising MongoDB, Express.js, React, and Node.js, with TypeScript applied across both the client and server for type safety. The system follows a layered, role-based architecture supporting three principal user classes: Tenant, Landlord, and Administrator. Geospatial querying, JSON Web Token authentication, role-based access control, cloud-based media storage, and automated PDF generation form the technical backbone of the platform.

The expected contribution of Domavi is a safer, more affordable, and more transparent rental experience for an underserved segment of the market. By reducing dependence on informal brokers, mitigating the risk of fraud, and replacing the roommate "gamble" with data-driven compatibility matching, the platform delivers measurable social value while remaining commercially

viable. This document presents the complete software requirements specification and software design for the system, providing the formal foundation for its implementation, testing, and evaluation.

Abbreviations

Abbreviation	Full Form
SRS	Software Requirements Specification
SDD	Software Design Document
API	Application Programming Interface
UI	User Interface
UX	User Experience
DB	Database
MERN	MongoDB, Express.js, React, Node.js
CNIC	Computerized National Identity Card
JWT	JSON Web Token
RBAC	Role-Based Access Control
REST	Representational State Transfer
CRUD	Create, Read, Update, Delete
HTTP/HTTPS	Hypertext Transfer Protocol (Secure)
PDF	Portable Document Format
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case
ERD	Entity Relationship Diagram
DFD	Data Flow Diagram
HEC	Higher Education Commission

Table of Contents

List of Tables

List of Figures

Chapter 1
Introduction

1.1 Introduction

Access to affordable and trustworthy rental accommodation is a fundamental need for the large population of students and young professionals who migrate to urban centres across Pakistan every year. Cities such as Multan, Lahore, and Islamabad host major universities and employment hubs that attract thousands of individuals who require short-term, low-cost living arrangements close to their place of study or work. For this segment, the ideal accommodation is rarely an entire house; instead it is a single room, a shared room, a bed space, or a hostel seat that fits a limited student or entry-level budget.

The current rental ecosystem, however, is poorly suited to this demand. The process is dominated by informal property dealers and word-of-mouth referrals. Prospective tenants typically rely on local brokers who charge substantial commissions, on unstructured social media groups where listings are unverified, or on classified websites that are designed primarily for full-house and commercial rentals. None of these channels offers a reliable mechanism to confirm that a listed property genuinely exists, that the advertised landlord is the true owner, or that a tenant is who they claim to be.

A second, equally significant problem is the absence of any systematic approach to roommate selection. Because shared accommodation is inherently a social arrangement, an incompatible roommate can transform an affordable living solution into a source of constant stress and conflict. At present, roommate selection is left almost entirely to chance, with users having no structured way to evaluate compatibility in terms of habits, study or work routines, cleanliness, and budget expectations.

These shortcomings result in high financial cost, wasted time, exposure to fraud, and avoidable interpersonal conflict. They also represent a clear gap in the digital marketplace: while ride-hailing, food delivery, and e-commerce have all been transformed by dedicated platforms, the micro-rental and roommate matching domain remains fragmented and underserved.

The proposed system, **Domavi**, is a web-based platform designed to close this gap. Domavi unifies a dedicated micro-rental marketplace, an intelligent roommate matching engine, a multi-layer trust and verification system, commute-based geospatial search, and end-to-end digital process automation within a single application. By bringing structure, transparency, and data-driven decision support to a previously informal process, Domavi aims to make shared housing safer, cheaper, and significantly easier to navigate for its target users.

1.2 Vision Statement

The vision of Domavi is to become the most trusted digital platform for affordable shared accommodation in Pakistan, serving primarily students and young professionals who seek single rooms, bed spaces, and hostel accommodation near their universities and workplaces. The platform seeks to eliminate the dependence on informal brokers and unverified listings by establishing a transparent, verification-driven marketplace where every property and every participant is authenticated.

Beyond simply listing rooms, Domavi aspires to redefine the shared-living experience by intelligently matching individuals with compatible roommates based on lifestyle, preferences, budget, and routine. In doing so, the platform addresses both the economic dimension of housing affordability and the social dimension of living harmony. The long-term vision is a safer, more affordable, and more transparent rental culture in which a student arriving in a new city can secure verified accommodation and a compatible roommate within minutes, entirely online, without paying excessive commissions or risking fraud. The uniqueness of Domavi lies in the combination of micro-rental focus, algorithmic roommate compatibility, and layered trust verification within one ecosystem, a combination that no existing local platform currently offers.

1.3 Related System Analysis

Several existing platforms partially address the housing search problem, but none of them target the specific micro-rental and roommate matching needs of students and young professionals in the Pakistani context. This section critically examines representative systems and contrasts them with the proposed solution.

General classified marketplaces such as OLX and similar local listing websites allow individuals to post rental advertisements. While they offer wide reach, they suffer from major weaknesses. Listings are entirely unverified, which exposes users to fraudulent advertisements and fake landlords. The platforms are general-purpose and are not optimised for single rooms or bed spaces, so users must manually sift through large volumes of irrelevant full-house and commercial listings. There is no roommate matching capability, no ownership verification, and no structured booking or agreement workflow.

Dedicated property portals such as Zameen.com focus on property buying, selling, and full-house renting. They provide professional listings and map views but are oriented toward higher-value transactions and full properties rather than affordable single-room rentals. They do not support roommate matching, lifestyle-based compatibility, or the kind of low-cost, short-term arrangements that students require.

International roommate platforms such as Roomster and SpareRoom demonstrate the value of roommate-oriented features and are useful reference points. However, they are designed for Western markets, often operate behind paywalls, lack localised verification suited to Pakistani identity documents such as the CNIC, and are not commonly used or trusted within the local market. Their compatibility features are also typically limited to basic preference filters rather than a weighted, multi-factor compatibility score.

Informal social media groups, particularly on Facebook and WhatsApp, are widely used in practice. They are free and community-driven but completely unstructured. There is no verification, no search or filtering, no compatibility assessment, and no protection against scams, making them unreliable for serious accommodation decisions.

In contrast, Domavi is purpose-built for the micro-rental segment. It combines verified listings, a multi-layer trust system tailored to local identity documents, a weighted roommate compatibility engine, commute-based proximity search, and an automated booking and agreement workflow. The comparative analysis below summarises the key weaknesses of existing solutions and the corresponding improvements offered by Domavi.

Table 1.1: Comparative Analysis of Existing Systems

Application Name	**Identified Weaknesses**	**Proposed Improvements in Domavi**
OLX / General Classifieds	• Unverified, fraud-prone listings • Not optimised for single rooms • No roommate matching	• Multi-layer verification • Dedicated micro-rental marketplace • Algorithmic roommate matching
Zameen.com	• Focused on full houses / sales • High-value, not student budget • No compatibility features	• Single rooms and bed spaces • Affordable, student-oriented • Lifestyle-based matching
Roomster / SpareRoom	• Designed for Western markets • Paywalled features • No local CNIC verification	• Localised for Pakistan • Free core functionality • CNIC and ownership verification
Facebook / WhatsApp Groups	• Completely unstructured • No verification or filtering • No fraud protection	• Structured search and filters • Verified users and listings • Secure, auditable workflow

1.4 Project Deliverables

The project produces a set of tangible software and documentation artifacts that collectively demonstrate a complete, working micro-rental and roommate matching platform. The principal deliverables are described below.

The core deliverable is a fully functional, responsive web application built on the MERN stack, accessible to tenants, landlords, and administrators through role-specific interfaces. Supporting this application is a documented backend REST API that exposes secure endpoints for authentication, property management, search, booking, payments, messaging, roommate matching, and administration. A structured MongoDB database, designed to store users, properties, roommate profiles, bookings, agreements, payments, conversations, messages, documents, and landlord bank accounts, underpins the entire system.

In addition to the running software, the project delivers complete engineering documentation, including this Software Requirements Specification and Software Design Document, covering use case models, functional and non-functional requirements, architectural and database design, and test cases. The deliverables also include the implemented feature modules that give the platform its distinctive value.

List of Deliverables:

1. Micro-Rental Marketplace – listing, searching, and management of single rooms, bed spaces, and hostels with photos, pricing, and amenities.
2. Roommate Matching Engine – a weighted compatibility scoring system that ranks potential roommates by lifestyle, preferences, budget, location, and schedule.
3. Trust and Verification Module – multi-layer verification including email/phone confirmation, administrator-reviewed CNIC verification, and property ownership validation.
4. Commute-Based Geospatial Search – Google Maps integration enabling proximity-based discovery of accommodation near a chosen university or workplace.
5. Booking, Payment, and Digital Agreement Workflow – structured booking requests, online payment recording with administrative confirmation, and auto-generated digital rental agreements in PDF form.
6. Secure Messaging Module – real-time, in-app conversations between tenants and landlords.
7. Administrative Dashboard – user management, verification review, payment oversight, and platform analytics.
8. Project Documentation – SRS, SDD, test documentation, and source code repository.

1.5 System Limitations / Constraints

While Domavi delivers a comprehensive solution, its scope is bounded by a number of technical and practical constraints that are important to acknowledge.

- **Online connectivity dependence** – The platform is a web application and requires a stable internet connection; it does not provide offline functionality.
- **Third-party service reliance** – Core features such as map-based search, media storage, and email delivery depend on external services (Google Maps API, cloud storage, and SMTP providers); availability and quota limits of these services constrain the system.
- **Manual verification overhead** – CNIC and ownership verification require administrator review, which introduces a human-in-the-loop step and limits the speed of fully automated onboarding.
- **Geographic focus** – The initial release targets selected Pakistani cities and university areas rather than nationwide coverage.
- **Payment handling** – The system records and confirms payments and bank-transfer details rather than executing live transactions through an integrated payment gateway in the initial version.
- **Resource and timeline constraints** – As an academic Final Year Project developed by a two-member team within a fixed academic timeline, certain advanced features are scoped for future work rather than the initial release.

1.6 Tools and Technologies

The technology choices for Domavi were guided by the need for rapid development, a single-language full-stack workflow, strong support for real-time and geospatial features, and proven scalability. The platform is built on the MERN stack, which allows JavaScript and TypeScript to be used uniformly across the client and server, reducing context-switching and improving maintainability. MongoDB was selected as the database because its flexible document model and native geospatial indexing are well suited to evolving listing schemas and proximity-based search. React with the Vite build tool provides a fast, component-driven frontend, while Node.js and Express.js power a modular REST API. Supporting libraries provide authentication, media storage, mapping, and document generation.

Table 1.2: Tools and Technologies Used in the Project

Tool / Technology	**Category**	**Purpose**
React + Vite	Frontend	Component-based, responsive user interface and fast development build
TypeScript	Frontend / Backend	Static typing for safer, more maintainable code on both ends
Tailwind CSS	Frontend	Utility-first styling and consistent, responsive design
Zustand	Frontend	Lightweight client-side state management
Node.js + Express.js	Backend	REST API and server-side business logic
MongoDB + Mongoose	Database	Document data store with schema modelling and geospatial indexing
JSON Web Token (JWT)	Security	Stateless authentication and session handling
Google Maps API	Integration	Commute-based search, geocoding, and map display
Cloud Media Storage	Integration	Storage of property images and verification documents
SMTP / Email Service	Integration	Email verification and notifications
PDF Generation Library	Backend	Auto-generation of digital rental agreements
Git / GitHub	Tooling	Version control and collaboration

1.7 Relevance to Course Modules

The development of Domavi integrates knowledge drawn from across the BS Information Technology curriculum, demonstrating the practical application of theoretical concepts learned throughout the degree program.

1.7.1 Programming Fundamentals

The project applies core programming principles including modular function design, control flow, exception handling, and object-oriented concepts. The codebase is organised into reusable services and controllers on the backend and composable components on the frontend, reflecting sound structuring, encapsulation, and separation of concerns learned in introductory programming courses.

1.7.2 Data Structures and Algorithms

The roommate matching engine is the clearest application of algorithmic knowledge. It computes a weighted compatibility score across multiple categories, sorts candidate profiles by score to return top matches, and caches previously computed scores for efficiency. Searching, filtering, and ranking of large listing sets also rely on appropriate data-structure and algorithmic choices.

1.7.3 Database Management Systems

Database design concepts such as schema modelling, relationships between collections, indexing, and query optimisation are applied throughout. The MongoDB data model defines structured collections for users, properties, bookings, payments, and agreements, with geospatial and compound indexes used to support efficient proximity search and status-based queries.

1.7.4 Software Engineering

The project follows a complete software engineering lifecycle. Requirement engineering, use case modelling, UML diagramming, architectural design, iterative development, and systematic testing are all applied. This very document, comprising the SRS and SDD, is a direct product of software engineering methodology, while version control and modular architecture reflect professional development practice.

1.7.5 Other Relevant Courses

Web technologies underpin the entire client-server implementation, including REST API design and responsive frontend development. Computer networking concepts inform the use of HTTP/HTTPS and secure communication. Human-computer interaction principles guide the user interface and experience design, while information security concepts shape the authentication, role-based access control, and multi-layer verification mechanisms.

1.8 Chapter Summary

This chapter introduced the background and context of the Domavi platform, highlighting the high cost, fraud risk, and roommate incompatibility that characterise the current micro-rental market for students and young professionals in Pakistan. The vision and strategic objectives of the system were outlined to establish its intended impact and innovation. A comparative analysis of related systems identified key limitations in current solutions such as general classifieds, property portals,

international roommate platforms, and informal social media groups. Project deliverables, system constraints, selected technologies, and academic relevance were also discussed.

This chapter establishes the foundation for the detailed problem definition presented in the next chapter.

Chapter 2
Problem Definition

2.1 Problem Statement

Students and young professionals seeking affordable shared accommodation in Pakistani cities face a rental market that is informal, opaque, and unsafe. Finding a single room or bed space near a university or workplace typically requires engaging local brokers who charge high commissions, or browsing unverified listings on general classified sites and social media groups where fraudulent advertisements and fake landlords are common. There is no reliable mechanism to confirm that a property exists, that the advertiser owns it, or that the parties involved are genuine. Compounding this, the selection of a roommate, an inherently social decision, is left entirely to chance, frequently resulting in conflict and stress. The affected stakeholders are tenants, who bear financial and security risks; genuine landlords, who lack a trustworthy channel to reach reliable tenants; and the wider community, which is denied a transparent housing marketplace. The problem is significant because it imposes avoidable financial cost, wasted time, exposure to fraud, and reduced quality of life on a large and growing population.

2.2 Proposed Solution Overview

Domavi is a web-based micro-rental and roommate matching platform that directly addresses the weaknesses of the current system. It provides a dedicated marketplace for single rooms, bed spaces, and hostels, replacing scattered and irrelevant listings with a focused, well-structured catalogue. To establish trust, the platform enforces a multi-layer verification system covering email and phone confirmation, administrator-reviewed CNIC verification, and property ownership validation, thereby reducing fraud. An intelligent roommate matching engine scores potential roommates on lifestyle, preferences, budget, location, and schedule, transforming roommate selection from guesswork into a data-driven decision. Commute-based search powered by the Google Maps API allows users to find accommodation within walking distance of their destination. Finally, secure in-app messaging, structured booking requests, payment recording, and auto-generated digital rental agreements automate the rental lifecycle. The result is a safer, cheaper, faster, and more transparent rental experience.

2.3 Objectives of the Proposed System

The objectives of Domavi are defined to be specific, measurable, and directly aligned with the problems identified above. Each objective begins with an action verb and is achievable within the project constraints.

Business Objectives (BO):

- BO-1: Develop a dedicated micro-rental marketplace that enables users to list, search, and book single rooms, bed spaces, and hostels.

- BO-2: Provide an intelligent roommate matching engine that ranks potential roommates by multi-factor compatibility to reduce living conflicts.
- BO-3: Ensure trust and safety through a multi-layer verification system covering identity, contact, and property ownership.
- BO-4: Improve accommodation discovery by offering commute-based geospatial search relative to universities and workplaces.
- BO-5: Automate the rental workflow through secure messaging, structured bookings, payment recording, and auto-generated digital rental agreements.
- BO-6: Provide administrators with effective tools to verify users, oversee payments, and monitor platform activity.

2.4 Scope of the System

Domavi is a responsive web application targeting students and young professionals seeking shared accommodation, the landlords who offer such accommodation, and the administrators who govern the platform. The system includes user registration and role-based access for tenants, landlords, and administrators; creation and management of room, bed-space, and hostel listings with images, pricing, amenities, and location; verification of users and properties; commute-based and filter-based search; roommate profile creation and compatibility-based matching; in-app messaging; booking requests and status management; payment recording and confirmation; auto-generated digital rental agreements; and an administrative dashboard with analytics.

The system explicitly excludes full-house sales and commercial property transactions, live integration with a third-party payment gateway in the initial release, native mobile applications, and nationwide coverage beyond the initially targeted cities. These exclusions delineate the platform's coverage and are distinct from the operational limitations described in Chapter 1.

2.5 System Architecture and Modules

Domavi follows a layered, client-server architecture organised into a presentation layer, an application (business logic) layer, and a data layer. The presentation layer is a React single-page application that communicates with the backend exclusively through a RESTful API. The application layer, built on Node.js and Express.js, hosts modular controllers and services that implement business rules and enforce role-based access control. The data layer is a MongoDB database accessed through Mongoose models. External services, including the Google Maps API, cloud media storage, and an email service, are integrated through the application layer. This modular design promotes loose coupling, reusability, and independent development and testing of each module, and a high-level architecture diagram is presented in Chapter 4.

2.5.1 Module 1: User and Profile Management

This module handles registration, authentication, authorisation, and profile management for the three supported user roles: Tenant, Landlord, and Administrator. It issues and validates JWT-

based sessions, enforces role-based access control, manages account status (pending, active, suspended), and supports email/phone confirmation and CNIC submission as part of the verification process.

Functional Requirements:

- FE-1: The system shall allow users to register using valid credentials and a selected role.
- FE-2: The system shall authenticate users and issue a secure session token before granting access.
- FE-3: The system shall allow users to view and update their profile information.
- FE-4: The system shall enforce role-based access to protected resources.

2.5.2 Module 2: Communication Module

This module enables secure, in-app interaction between tenants and landlords. It supports conversation threads, real-time message exchange, and notification of relevant events such as booking updates, allowing parties to negotiate and coordinate without sharing personal contact details prematurely.

Functional Requirements:

- FE-1: The system shall enable secure messaging between tenants and landlords.
- FE-2: The system shall generate notifications for booking and verification events.
- FE-3: The system shall maintain a persistent history of conversations and messages.

2.5.3 Module 3: Property Listing and Search Module (Core)

This module delivers the core value of the platform. It allows landlords to create and manage micro-rental listings and allows tenants to search and filter those listings, including commute-based proximity search using the Google Maps API. It directly addresses the discovery problem identified in Section 2.1.

Functional Requirements:

- FE-1: The system shall allow landlords to create, update, and remove property listings.
- FE-2: The system shall allow tenants to search and filter listings by location, price, room type, and gender preference.
- FE-3: The system shall support commute-based search returning listings near a chosen location.
- FE-4: The system shall display only verified and active listings in public search results.

2.5.4 Module 4: Booking, Payment, and Agreement Module

This module manages the rental transaction lifecycle. Tenants submit booking requests, landlords accept or reject them, payments are recorded and confirmed, and a digital rental agreement is auto-generated as a PDF for the confirmed booking.

Functional Requirements:

- FE-1: The system shall allow tenants to submit booking requests for available listings.
- FE-2: The system shall allow landlords to accept or reject booking requests.
- FE-3: The system shall record payment details and update payment status upon confirmation.
- FE-4: The system shall auto-generate a digital rental agreement for confirmed bookings.

2.5.5 Module 5: Roommate Matching Module (Advanced)

This module provides the platform's distinctive intelligent capability. Users complete a lifestyle and preference profile, and the matching engine computes a weighted compatibility score against other profiles, returning a ranked list of the most compatible potential roommates.

Functional Requirements:

- FE-1: The system shall allow users to create and update a roommate lifestyle profile.
- FE-2: The system shall compute a weighted compatibility score between roommate profiles.
- FE-3: The system shall return a ranked list of top roommate matches for a given profile.

2.5.6 Module 6: Administration Module

This module gives administrators control over the platform. It supports reviewing and approving CNIC and ownership verification requests, managing user accounts and status, overseeing payments, and viewing platform analytics.

Functional Requirements:

- FE-1: The system shall allow administrators to review and approve or reject verification requests.
- FE-2: The system shall allow administrators to manage user accounts and account status.
- FE-3: The system shall provide administrators with analytics on users, listings, bookings, and payments.

2.6 Assumptions and Dependencies

The design of Domavi rests on a number of assumptions and external dependencies that define its operating conditions.

- The system assumes that users have access to a stable internet connection and a modern web browser.
- The system assumes that users possess basic digital literacy and provide truthful information during verification.
- The system depends on the continued availability and quota limits of the Google Maps API for geospatial features.
- The system depends on third-party cloud media storage for images and verification documents, and on an SMTP email service for notifications.
- The system requires administrator availability to process manual CNIC and ownership verification requests in a timely manner.

2.7 Chapter Summary

This chapter defined the core problem addressed by Domavi and outlined the conceptual solution approach. Clear, measurable objectives were established to ensure alignment with the identified challenges of cost, fraud, discovery, and roommate incompatibility. The system scope and architectural overview were presented to define structural boundaries and major modules, namely user and profile management, communication, property listing and search, booking and agreements, roommate matching, and administration. Key assumptions and dependencies were also documented to clarify operational constraints.

This chapter provides the problem foundation and sets the direction for the detailed requirement analysis and software design presented in the following chapters.

Chapter 3
Requirement Analysis

3.1 Introduction to Requirement Analysis

This chapter presents a structured and formal specification of the system requirements for the Domavi platform. The purpose of this chapter is to clearly define what the system shall accomplish before proceeding to the system design phase. The requirements documented here serve as a contractual foundation for implementation, validation, and system testing.

The requirement analysis includes requirement elicitation techniques, identification of user classes, use case modelling, functional requirements, non-functional requirements, and external interface requirements. All requirements are written in measurable and testable form using standard requirement engineering principles.

3.2 Requirement Elicitation Techniques

This section describes the techniques used to gather and refine the requirements of the Domavi platform. Requirement elicitation ensured that the platform's capabilities are aligned with the real needs of students, young professionals, and landlords.

Requirements for this project were identified using the following techniques:

- Informal interviews and discussions with students and young professionals about their experiences finding rooms and roommates.
- Analysis of existing solutions such as OLX, Zameen.com, international roommate platforms, and social media housing groups to identify functional gaps.
- Literature review of micro-rental and roommate matching systems and their design approaches.
- Observation of the current, broker-driven rental workflow to understand pain points around cost, verification, and trust.
- Brainstorming and supervisor feedback sessions to refine and prioritise the proposed feature set.

3.3 User Classes and Characteristics

This section identifies the different categories of users who interact with the Domavi platform. Each user class is defined along with its responsibilities, access privileges, and technical proficiency. This classification supports the role-based access control enforced by the system.

Table 3.1: User Classes and Characteristics

User Class	Description	Technical Level
Tenant	Searches for rooms and roommates, submits booking requests, makes payments, messages landlords, and completes a roommate profile. The primary end user of the platform.	Basic
Landlord	Lists and manages rooms, bed spaces, and hostels, submits ownership and identity documents for verification, and accepts or rejects booking requests.	Basic to Intermediate
Administrator	Reviews and approves verification requests, manages user accounts and status, oversees payments, and monitors platform analytics.	Advanced
External Service	Third-party systems integrated with the platform, including the Google Maps API, cloud media storage, and the email service.	Technical

3.4 System Overview

Domavi operates within a layered, client-server architecture designed to support secure, role-based interaction between user interfaces, business logic, and data storage. The React-based presentation layer communicates with a Node.js and Express.js application layer through a RESTful API, and the application layer persists data in a MongoDB database via Mongoose models. The system boundary encompasses all internal modules, namely user and profile management, property listing and search, roommate matching, booking, payment, agreement generation, messaging, and administration, together with the integrated external services for mapping, media storage, and email. Role-based access control governs every protected interaction, ensuring that tenants, landlords, and administrators can only perform actions appropriate to their role.

3.5 Use Case Model

The use case model represents the high-level interactions between the actors and the functionalities of the Domavi platform. It defines the system boundary and captures functional behaviour from the perspective of each user.

3.5.1 Use Case Diagram

The diagram above illustrates the interaction between the three primary actors, Tenant, Landlord, and Administrator, and the major system functionalities. Common functionality such as registration, login, and verification is shared through *include* relationships, while optional behaviour such as roommate matching extends the core search experience. *(This is a sample diagram from the template; replace it with the finalised Domavi use case diagram before submission.)*

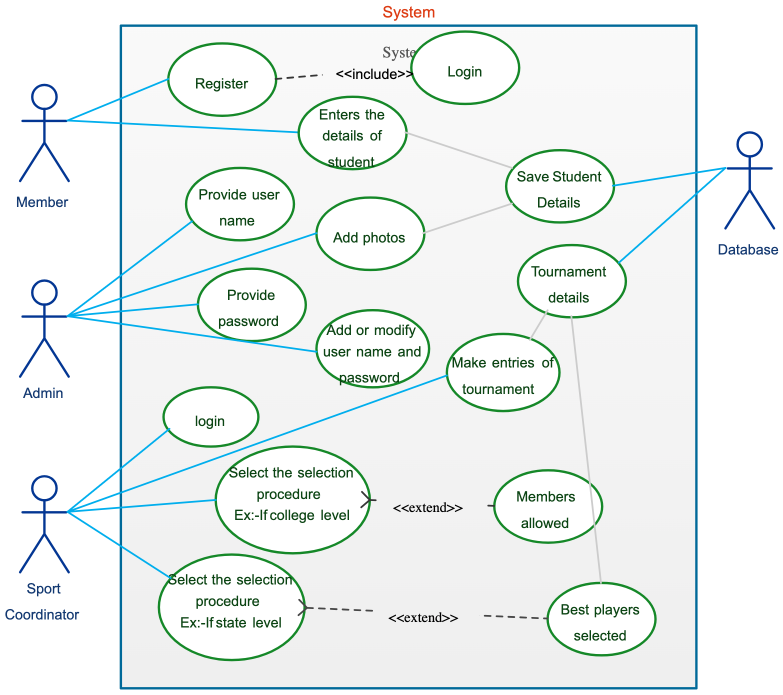

Figure 3.1: Use Case Diagram of the Domavi Platform

3.5.2 Use Case Descriptions

This section provides detailed tabular specifications of the major use cases. These use cases form the basis for deriving the functional requirements presented later in this chapter.

3.5.2.1 Module 1: User and Verification

Below are detailed use cases for registration and verification.

3.5.2.2 Module 2: Property and Search

3.5.2.3 Module 3: Booking and Roommate Matching

3.5.3 Event-Response Table

The event-response table defines how the system reacts to significant internal and external events.

3.6 Functional Requirements

Functional requirements define the core services and behaviours that the system shall provide. These requirements are derived directly from the use case model presented in Section 3.5. Each requirement is uniquely identified, measurable, and written using formal specification language, and the requirements are categorised by system module.

Table 3.2: UC-1.1: User Registration

Use Case ID	UC-1.1
Use Case Name	User Registration
Actors	Tenant, Landlord
Description	A new user creates an account and selects a role (tenant or landlord).
Trigger	User selects the "Sign Up" option.
Preconditions	User is not already registered with the provided email.
Postconditions	A new account is created with status *pending* and a verification email is sent.
Normal Flow	1. User enters name, email, password, and role. 2. System validates the input fields. 3. System creates the account and stores the hashed password. 4. System sends an email verification link. 5. Confirmation message is displayed.
Alternative Flow	If the email already exists or validation fails, the system displays an appropriate error message.

3.6.1 Module 1: User and Verification Management

3.6.1.1 FR-1.1 User Registration

3.6.1.2 FR-1.2 Multi-Layer Verification

3.6.2 Module 2: Property Listing and Search

3.6.2.1 FR-2.1 Manage Property Listings

3.6.2.2 FR-2.2 Commute-Based Search

3.6.3 Module 3: Booking, Payment, and Agreements

3.6.3.1 FR-3.1 Booking Management

3.6.3.2 FR-3.2 Digital Rental Agreement

3.6.4 Module 4: Roommate Matching

3.6.4.1 FR-4.1 Roommate Compatibility Matching

3.6.5 Module 5: Communication and Administration

3.6.5.1 FR-5.1 Secure Messaging

3.6.5.2 FR-5.2 Administration and Analytics

3.7 Non-Functional Requirements

Non-functional requirements define the quality attributes that describe how the system performs. These requirements are measurable and support reliability, usability, performance, security, and

Table 3.3: UC-1.2: CNIC and Ownership Verification

Use Case ID	UC-1.2
Use Case Name	CNIC and Ownership Verification
Actors	Landlord, Administrator
Description	A landlord submits identity and ownership documents, which an administrator reviews and approves or rejects.
Trigger	Landlord uploads CNIC and ownership proof.
Preconditions	Landlord is registered and logged in.
Postconditions	The verification request is approved or rejected and the user/property status is updated accordingly.
Normal Flow	1. Landlord uploads required documents. 2. System stores the documents securely and creates a verification request. 3. Administrator reviews the documents. 4. Administrator approves or rejects the request. 5. System updates the verification status and notifies the landlord.
Alternative Flow	If the documents are invalid or unclear, the administrator rejects the request with a reason.

maintainability.

3.7.1 Performance Requirements

- PER-1: The system shall load the main dashboard and search results within 3 seconds under normal operating conditions.
- PER-2: The system shall return roommate match results within an acceptable latency for a typical candidate pool.
- PER-3: The system shall support multiple concurrent users as defined in the project scope without significant degradation.
- PER-4: Geospatial search queries shall be optimised through appropriate indexing to ensure responsive results.

3.7.2 Security Requirements

- SEC-1: The system shall implement secure authentication using JSON Web Tokens.
- SEC-2: Passwords shall be stored using a secure hashing algorithm with salting.

Table 3.4: UC-2.1: Create Property Listing

Use Case ID	UC-2.1
Use Case Name	Create Property Listing
Actors	Landlord
Description	A verified landlord creates a listing for a room, bed space, or hostel.
Trigger	Landlord selects "Add Property".
Preconditions	Landlord is logged in and verified.
Postconditions	A new listing is created with status *pending_verification*.
Normal Flow	1. Landlord enters title, description, type, price, amenities, and location. 2. Landlord uploads property images. 3. System validates and geocodes the location. 4. System saves the listing for review.
Alternative Flow	If required fields are missing, the system prompts the landlord to complete them.

- SEC-3: Sensitive data shall be transmitted over HTTPS.
- SEC-4: Role-based access control shall be enforced for every protected resource.
- SEC-5: Verification documents (CNIC, ownership proof) shall be stored securely and accessible only to authorised administrators.

3.7.3 Reliability Requirements

- REL-1: The system shall aim to maintain high availability with at least 99% uptime.
- REL-2: System failures shall not result in loss of booking, payment, or agreement data.
- REL-3: Backup and recovery mechanisms shall be in place for the database.

3.7.4 Usability Requirements

- USE-1: The interface shall be intuitive and easy to navigate for users with basic digital literacy.
- USE-2: Clear and meaningful error messages shall be displayed for invalid actions.
- USE-3: The system shall follow consistent UI/UX design principles across all pages.
- USE-4: The application shall be fully responsive across desktop and mobile browsers.

Table 3.5: UC-2.2: Commute-Based Property Search

Use Case ID	UC-2.2
Use Case Name	Commute-Based Property Search
Actors	Tenant
Description	A tenant searches for listings near a chosen university or workplace using filters.
Trigger	Tenant enters a location and search criteria.
Preconditions	Active and verified listings exist in the database.
Postconditions	A ranked list of nearby matching listings is displayed.
Normal Flow	1. Tenant enters a target location and filters (price, type, gender preference). 2. System geocodes the location via the Maps API. 3. System performs a geospatial query against active listings. 4. System returns and displays matching listings on a map and list.
Alternative Flow	If no listings match, the system displays an empty-state message with suggestions.

3.7.5 Scalability Requirements

- SCA-1: The system shall support a growing number of users and listings.
- SCA-2: The architecture shall allow horizontal or vertical scaling of the application and database.
- SCA-3: The database design shall support large volumes of listings and roommate profiles.

3.7.6 Maintainability Requirements

- MAIN-1: The system shall follow a modular architecture with clear separation of concerns.
- MAIN-2: Source code shall be written in TypeScript and documented for clarity.
- MAIN-3: Future features shall be integrable without major restructuring of existing modules.

3.8 External Interface Requirements

This section defines how the Domavi platform interacts with external entities including users, hardware devices, software systems, and communication networks.

Table 3.6: UC-3.1: Submit Booking Request

Use Case ID	UC-3.1
Use Case Name	Submit Booking Request
Actors	Tenant, Landlord
Description	A tenant requests to book a listing; the landlord accepts or rejects the request.
Trigger	Tenant selects "Request Booking".
Preconditions	Tenant is logged in and the listing is active.
Postconditions	A booking record is created with status *pending* and the landlord is notified.
Normal Flow	1. Tenant selects move-in date and submits the request. 2. System creates a booking with pending status. 3. Landlord reviews and accepts or rejects the request. 4. System updates the booking status and notifies the tenant.
Alternative Flow	If the landlord rejects the request, the tenant is notified and may search for alternatives.

3.8.1 User Interface Requirements

- UI-1: The system shall provide a graphical web interface accessible from modern browsers.
- UI-2: Standard, readable font sizes shall be used throughout (12–14pt body text).
- UI-3: The layout shall be responsive across desktop, tablet, and mobile devices.
- UI-4: Navigation shall remain consistent and role-appropriate across all modules.

3.8.2 Hardware Interface Requirements

- HI-1: The system shall operate on standard computing devices with an internet connection.
- HI-2: The application shall function on devices running modern web browsers without specialised hardware.
- HI-3: Server hosting shall provide sufficient compute and storage for the database and media.

3.8.3 Software Interface Requirements

- SI-1: The system shall interact with a MongoDB database through the Mongoose object data modelling layer.
- SI-2: The system shall integrate the Google Maps API for geocoding and map display.

Table 3.7: UC-3.2: Find Roommate Matches

Use Case ID	UC-3.2
Use Case Name	Find Roommate Matches
Actors	Tenant
Description	A tenant requests a ranked list of compatible roommates based on their lifestyle profile.
Trigger	Tenant opens the "Roommate Matches" page.
Preconditions	Tenant has completed a roommate lifestyle profile.
Postconditions	A ranked list of compatible roommate profiles is displayed.
Normal Flow	1. System retrieves the tenant's roommate profile. 2. System computes weighted compatibility scores against other profiles. 3. System sorts candidates by score. 4. System displays the top matches with their compatibility scores.
Alternative Flow	If the profile is incomplete, the system prompts the tenant to complete it first.

- SI-3: The system shall integrate cloud media storage for images and verification documents.
- SI-4: The system shall integrate an SMTP email service for verification and notifications.

3.8.4 Communication Interface Requirements

- CI-1: The system shall use HTTPS for secure client-server communication.
- CI-2: RESTful APIs shall be used for all backend communication.
- CI-3: Real-time messaging shall be supported for in-app conversations.
- CI-4: All API requests to protected endpoints shall carry a valid authentication token.

3.9 Assumptions and Dependencies

This section lists the assumptions made during requirement analysis and the external dependencies on which the system relies.

- The system assumes stable internet connectivity for all online operations.
- Third-party services (Google Maps API, cloud media storage, email service) are assumed to remain available and within quota.
- Users are assumed to provide truthful information and possess basic digital literacy.

Table 3.8: Event-Response Table

Event	Source	System Response
User Login Attempt	Tenant / Landlord / Admin	Validate credentials and grant or deny access based on role.
Document Upload	Landlord	Store CNIC/ownership document securely and create a verification request.
Property Search	Tenant	Geocode the location and return nearby active listings.
Booking Request	Tenant	Create a pending booking and notify the landlord.
Roommate Match Request	Tenant	Compute weighted compatibility scores and return ranked matches.
Payment Submission	Tenant	Record payment details and set status pending until admin confirmation.

Table 3.9: Functional Requirement – User Registration

Identifier	FR-1.1
Title	User Registration
Requirement	The system shall allow users to create an account by providing a name, valid email, password, and role (tenant or landlord).
Source	Stakeholder Requirement
Rationale	Enables authorised, role-based access to platform services.
Dependencies	Database connectivity, email service
Priority	High

- Administrators are assumed to be available to process manual verification requests in a timely manner.

3.10 Requirement Traceability Matrix

This matrix maps project objectives to use cases and functional requirements to ensure alignment and completeness.

Chapter Summary

This chapter presented a structured requirement analysis of the Domavi platform. It defined the requirement elicitation techniques, the user classes (tenant, landlord, administrator, and external services), and a use case model with detailed use case specifications for registration, verification, listing, search, booking, and roommate matching. Functional requirements were specified module by module, followed by measurable non-functional requirements covering performance, security, reliability, usability, scalability, and maintainability. External interface requirements and a traceability matrix linking objectives to requirements were also included. These requirements form the formal foundation for the system design presented in the next chapter.

Table 3.10: Functional Requirement – Multi-Layer Verification

Identifier	FR-1.2
Title	Multi-Layer Verification
Requirement	The system shall support email/phone confirmation and allow landlords to submit CNIC and ownership documents for administrator review.
Source	Business Objective BO-3
Rationale	Establishes trust and reduces fraud on the platform.
Dependencies	Media storage, administration module
Priority	High

Table 3.11: Functional Requirement – Manage Property Listings

Identifier	FR-2.1
Title	Manage Property Listings
Requirement	The system shall allow verified landlords to create, update, and remove listings for rooms, bed spaces, and hostels, including images, price, amenities, and location.
Source	Business Objective BO-1
Rationale	Provides the supply side of the micro-rental marketplace.
Dependencies	Media storage, Maps API (geocoding)
Priority	High

Table 3.12: Functional Requirement – Commute-Based Search

Identifier	FR-2.2
Title	Commute-Based Search
Requirement	The system shall allow tenants to search and filter active, verified listings by proximity to a chosen location, price, room type, and gender preference.
Source	Business Objective BO-4
Rationale	Enables efficient discovery of relevant accommodation near study or work.
Dependencies	Geospatial index, Maps API
Priority	High

Table 3.13: Functional Requirement – Booking Management

Identifier	FR-3.1
Title	Booking Management
Requirement	The system shall allow tenants to submit booking requests and allow landlords to accept or reject them, updating the booking status accordingly.
Source	Business Objective BO-5
Rationale	Structures and tracks the rental transaction.
Dependencies	Property module, notification service
Priority	High

Table 3.14: Functional Requirement – Digital Rental Agreement

Identifier	FR-3.2
Title	Digital Rental Agreement
Requirement	The system shall auto-generate a digital rental agreement in PDF format for each confirmed booking, recording the tenant, landlord, property, and terms.
Source	Business Objective BO-5
Rationale	Automates documentation and provides a verifiable record.
Dependencies	Booking module, PDF generation library
Priority	Medium

Table 3.15: Functional Requirement – Roommate Compatibility Matching

Identifier	FR-4.1
Title	Roommate Compatibility Matching
Requirement	The system shall compute a weighted compatibility score between roommate profiles across lifestyle, preferences, budget, location, and schedule, and return a ranked list of top matches.
Source	Business Objective BO-2
Rationale	Replaces the roommate "gamble" with data-driven matching.
Dependencies	Roommate profile data
Priority	High

Table 3.16: Functional Requirement – Secure Messaging

Identifier	FR-5.1
Title	Secure Messaging
Requirement	The system shall enable secure, in-app messaging between tenants and landlords and maintain a persistent conversation history.
Source	Stakeholder Requirement
Rationale	Allows coordination without prematurely exposing personal contact details.
Dependencies	Authentication module
Priority	Medium

Table 3.17: Functional Requirement – Administration and Analytics

Identifier	FR-5.2
Title	Administration and Analytics
Requirement	The system shall allow administrators to review verification requests, manage user accounts and status, oversee payments, and view platform analytics.
Source	Business Objective BO-6
Rationale	Provides governance, oversight, and operational insight.
Dependencies	All modules
Priority	High

Table 3.18: Requirement Traceability Matrix

Objective	**Use Case**	**Functional Requirement**
BO-1	UC-2.1	FR-2.1
BO-2	UC-3.2	FR-4.1
BO-3	UC-1.2	FR-1.2
BO-4	UC-2.2	FR-2.2
BO-5	UC-3.1	FR-3.1, FR-3.2
BO-6	UC-1.2	FR-5.2

Chapter 4
Software Design

4.1 Introduction to Software Design

This chapter presents the detailed software design of the Domavi platform. It translates the requirements identified in Chapter 3 into structured architectural, behavioural, and data models. The purpose of this chapter is to provide a clear blueprint for system implementation, ensuring maintainability, scalability, and modular development.

The chapter covers the design methodology and software process model, a high-level system overview and architecture, UML behavioural and structural models, data flow diagrams, and the database design including the entity relationship model, schema, and data dictionary for the platform's MongoDB collections.

4.2 Design Methodology and Software Process Model

4.2.1 Design Methodology

Domavi follows an object-oriented, layered design methodology combined with the Model-View-Controller pattern on the backend. The system is divided into a presentation layer (React client), an application layer (Express controllers and services), and a data layer (MongoDB through Mongoose models). Within the application layer, controllers handle request and response concerns while services encapsulate business logic, giving a clear separation of concerns.

This methodology was selected because it suits a web application with multiple user roles and evolving features. It enforces loose coupling between layers, promotes reuse of services across controllers, and allows each module to be developed and tested independently. The design adheres to established principles including Separation of Concerns, the DRY (Don't Repeat Yourself) principle, and SOLID object-oriented design guidelines, particularly single-responsibility services and dependency inversion through well-defined interfaces.

4.2.2 Software Process Model

The project follows the **Agile (iterative and incremental)** process model. Agile was selected because the requirements were refined progressively through stakeholder feedback and supervisor reviews, and because the platform's features, such as listing management, search, booking, and roommate matching, lend themselves naturally to incremental delivery. Development proceeded in short iterations, each delivering a working slice of functionality that was tested and reviewed before the next iteration began. Testing and validation were integrated continuously rather than deferred to a single final phase, allowing defects to be identified and corrected early.

4.3 System Overview

This section provides a high-level overview of the system components and their interactions with external entities. At the boundary, three human actors, the tenant, the landlord, and the administrator, interact with the Domavi platform, while three external services, the Google Maps API, cloud media storage, and the email service, support the platform's operation. The context diagram below depicts the system as a single process exchanging data with these external entities.

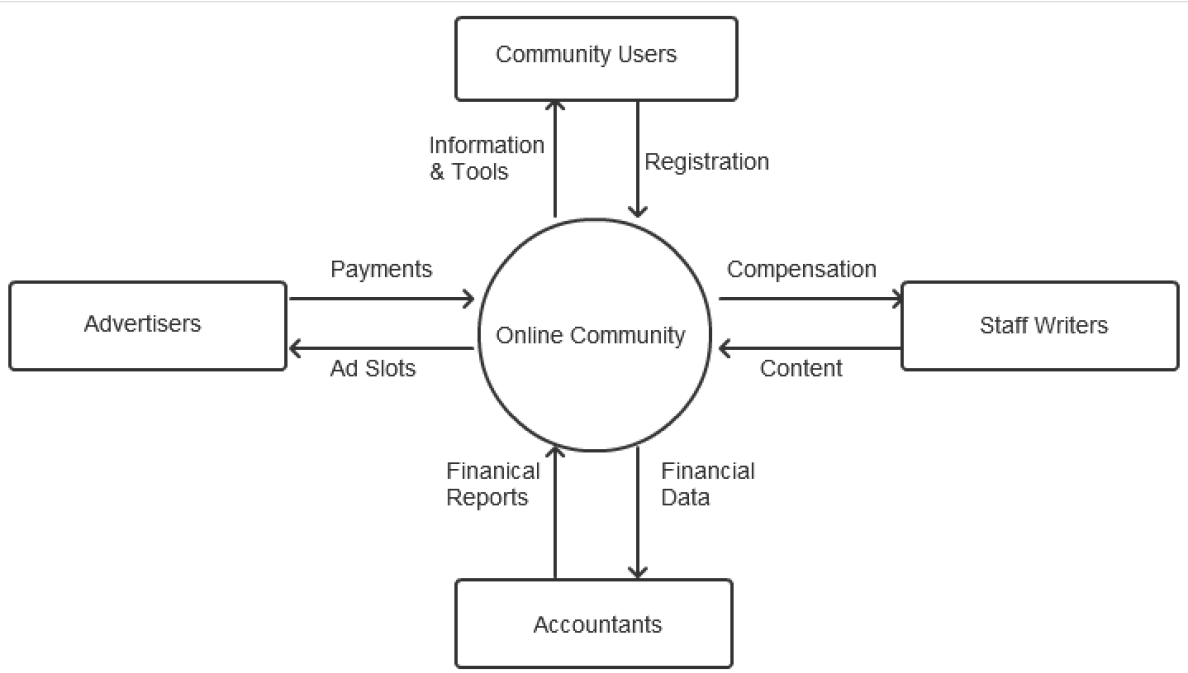

Figure 4.1: Context Diagram of the Domavi Platform

(The figure above is a sample from the template; replace it with the finalised Domavi context diagram before submission.) The diagram shows the system boundary, the human actors who submit requests and receive responses, and the third-party services that the platform calls for mapping, storage, and email.

4.4 Architectural Design

Domavi adopts a three-layer client-server architecture. The presentation layer is a React single-page application built with Vite and styled with Tailwind CSS, using Zustand for client-side state. It communicates with the backend solely through a RESTful API over HTTPS. The application layer is a Node.js and Express.js server organised into routes, controllers, and services; it enforces authentication and role-based access control through middleware and implements all business logic in dedicated service classes such as the authentication, property, booking, payment, chat, agreement, and roommate-matching services. The data layer is a MongoDB database accessed through Mongoose models, complemented by external integrations for geospatial search, media storage, and email.

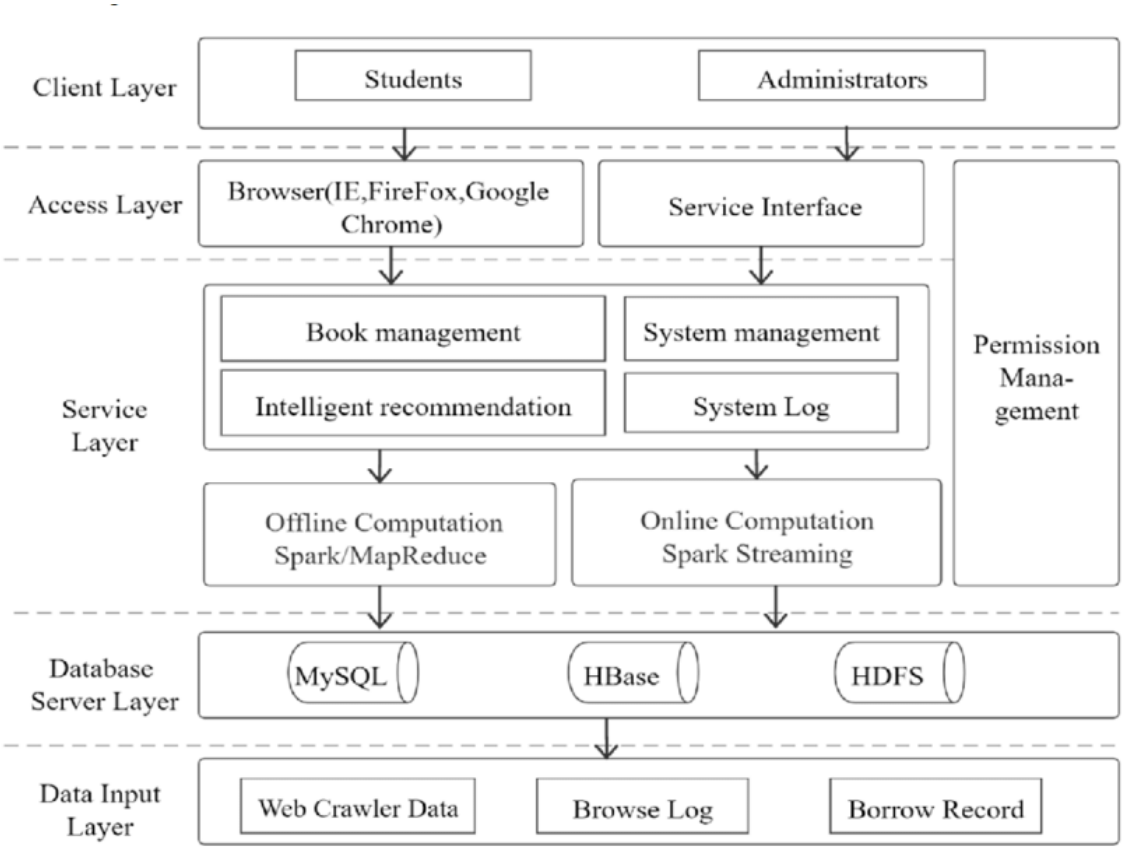

Figure 4.2: System Architecture of the Domavi Platform

(Sample template figure; replace with the finalised Domavi architecture diagram.) The presentation layer issues API requests; the API layer authenticates and authorises each request, delegates to the appropriate service, and the service interacts with the data layer and external services. Responses flow back through the same layers, keeping the frontend decoupled from data and integration concerns.

4.5 Design Models

This section presents detailed design models describing system behaviour, structure, and interactions using UML diagrams. These models help developers understand the system workflow before implementation.

4.5.1 Activity Diagrams

Activity diagrams represent the workflow of the major modules of the platform. Each core module is described by a separate activity diagram capturing user interaction, validation checks, decision points, and database interaction.

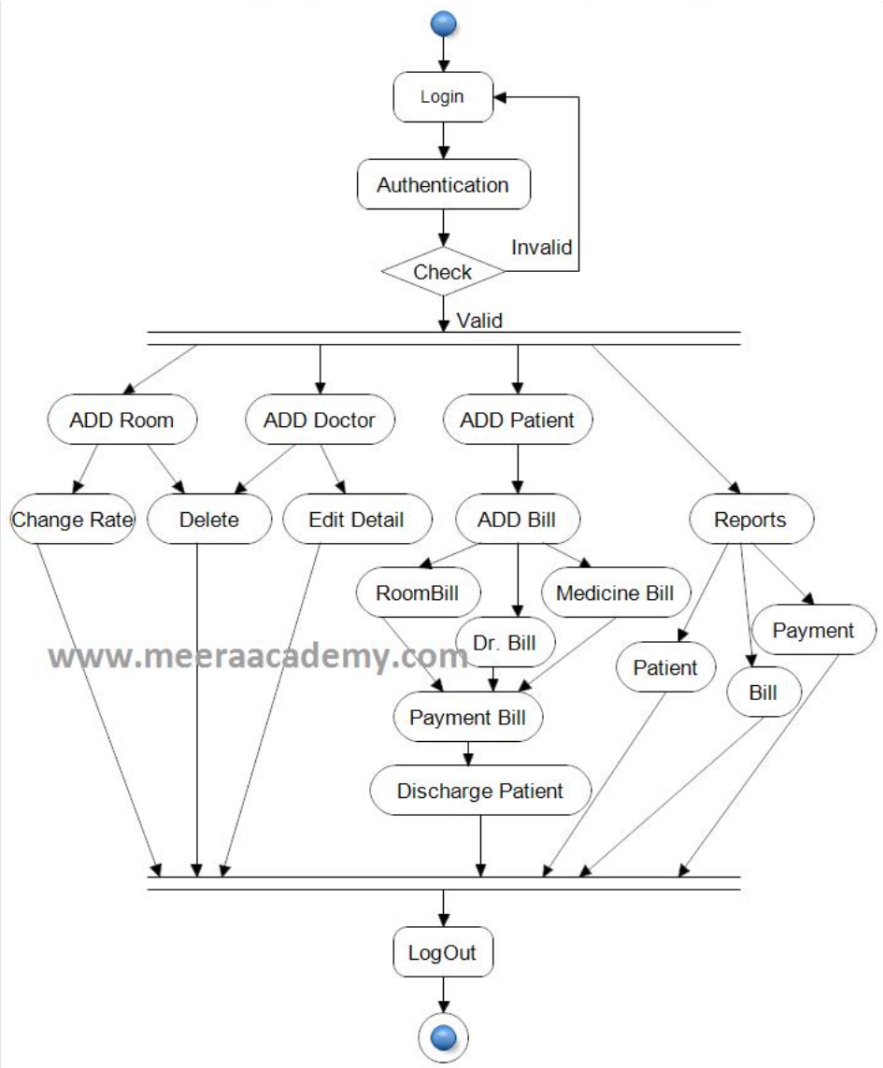

Figure 4.3: Activity Diagram (Sample)

4.5.1.1 Module 1: User Management

This activity diagram illustrates the registration, login, and verification workflow. The flow begins with the user submitting credentials, proceeds through input validation, account creation, and email verification, and includes a decision point where unverified accounts are restricted from protected actions until verification completes.

4.5.1.2 Module 2: Property Listing and Booking

This activity diagram captures the core rental workflow: a landlord creates a listing, the listing awaits verification, a tenant searches and submits a booking request, the landlord accepts or rejects it, payment is recorded, and a digital agreement is generated. Decision nodes handle validation failures, rejected bookings, and unconfirmed payments.

4.5.1.3 Module 3: Admin Module

This activity diagram describes administrator operations, including reviewing verification requests, approving or rejecting documents, managing user status, confirming payments, and viewing analytics.

4.5.2 Class Diagram

The class diagram shows the static structure of the system, including the principal domain classes, their attributes, and their relationships.

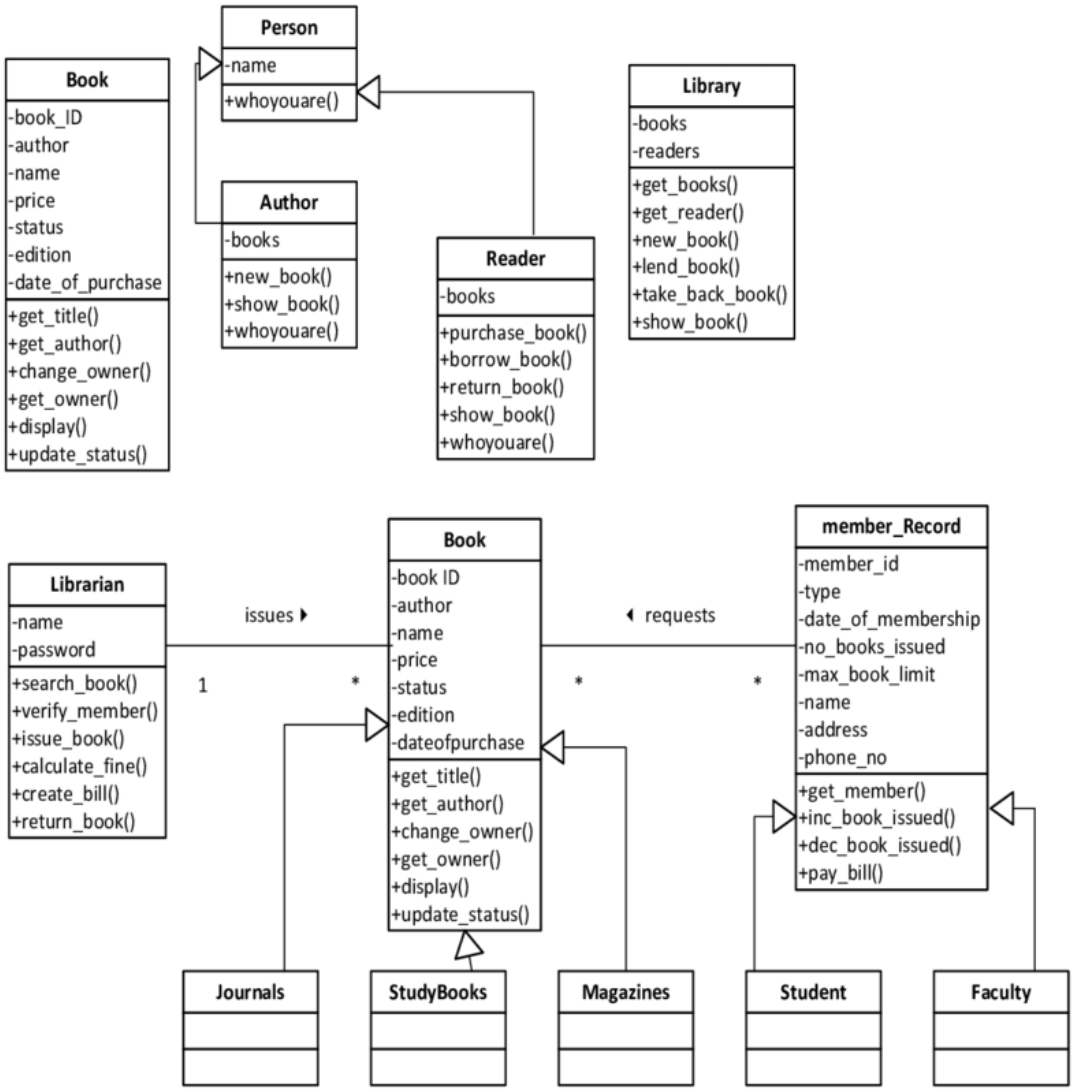

Figure 4.4: Class Diagram (Sample)

The key classes of Domavi are `User` (with subtypes distinguished by role), `Property`, `RoommateProfile`, `Booking`, `Agreement`, `Payment`, `Conversation`, `Message`, `Document`, and `LandlordBankAccount`. A `User` acting as a landlord owns many `Property` listings; a `User` acting as a tenant creates many

`Booking` records; each `Booking` relates one tenant, one landlord, and one property and may generate one `Agreement` and one or more `Payment` records. `Conversation` aggregates many `Message` objects between two users. Each `User` may have one `RoommateProfile`, which the matching service uses to compute compatibility. These relationships include association, aggregation, and role-based specialisation.

4.5.3 Sequence Diagrams

Sequence diagrams show the interaction between objects over time, including the request-response flow between the actor, the system components, and the database.

4.5.3.1 Sequence Diagram – User Login

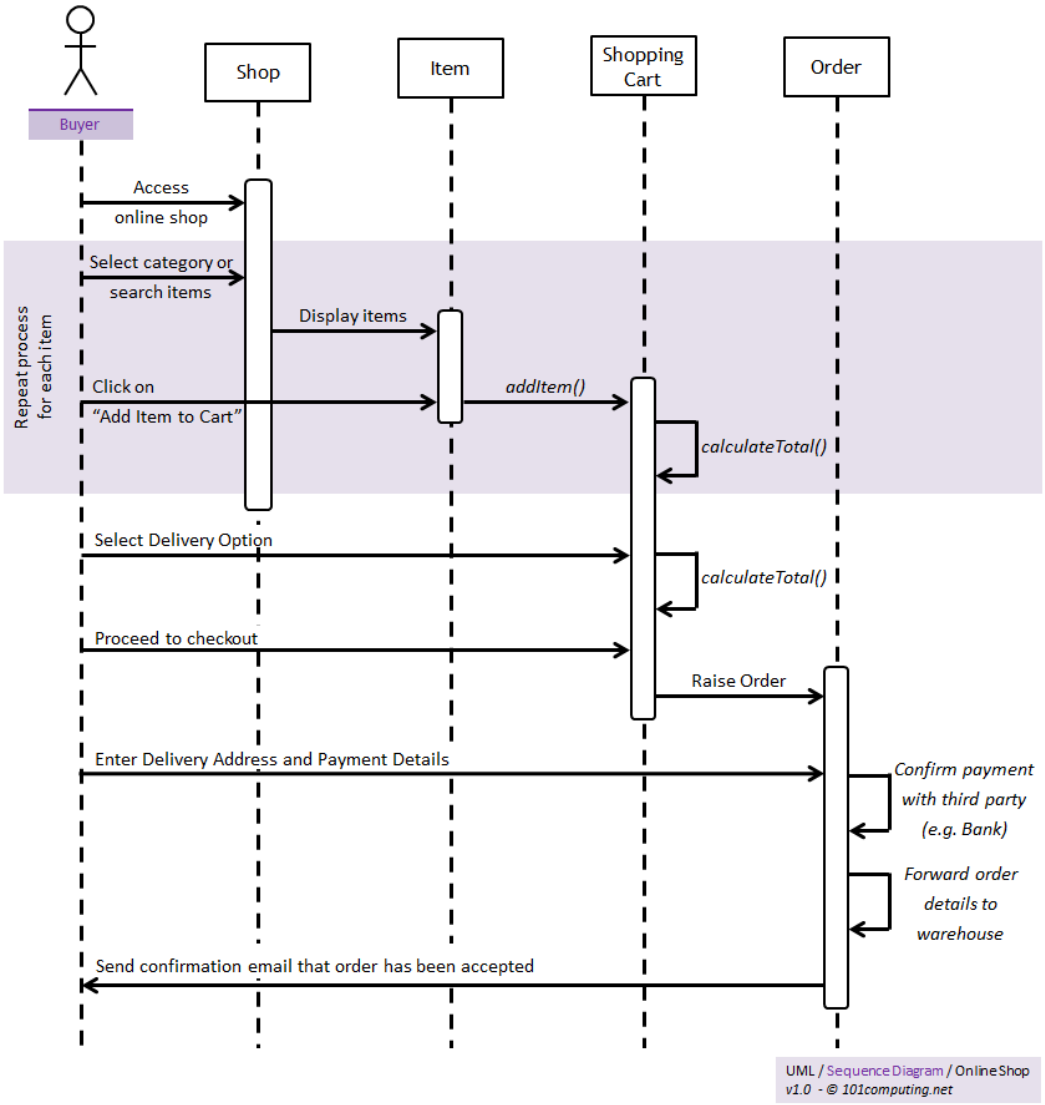

Figure 4.5: Sequence Diagram – User Login (Sample)

In the login sequence, the user submits credentials to the React client, which sends them to the authentication endpoint. The authentication service validates the credentials against the user record in MongoDB, verifies the hashed password, issues a JWT on success, and returns it to the client, which stores it for subsequent authenticated requests.

4.5.3.2 Sequence Diagram – Booking and Agreement

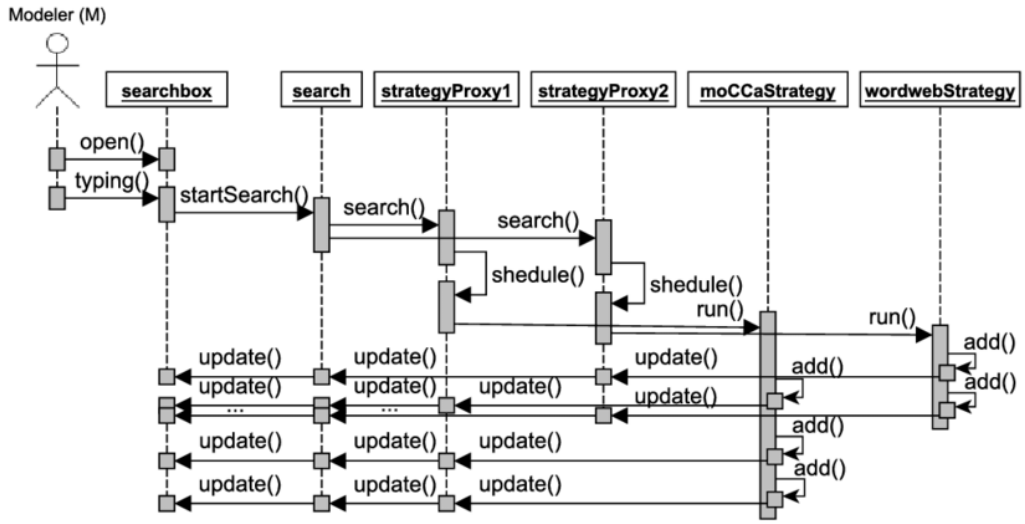

Figure 4.6: Sequence Diagram – Booking and Agreement (Sample)

In the booking sequence, the tenant submits a booking request; the booking service creates a pending booking and notifies the landlord; upon acceptance and confirmed payment, the agreement service generates a PDF rental agreement and persists it, after which both parties are notified.

4.5.4 State Transition Diagrams

State transition diagrams show how key entities change state over their lifecycle. Two important examples in Domavi are the user account and the booking. A **User account** transitions through the states *Pending* (just registered), *Active* (verified), and *Suspended* (administratively blocked). A **Booking** transitions through *Pending* (requested), *Accepted* or *Rejected* (landlord decision), *Confirmed* (payment recorded), and *Completed* or *Cancelled*. A **Property** listing transitions through *Pending Verification*, *Active*, and *Inactive*. Each transition is triggered by a specific event, such as approval, payment, or administrative action.

4.6 Data Flow Diagrams

Data Flow Diagrams describe how data moves within the system. Different levels provide increasing detail.

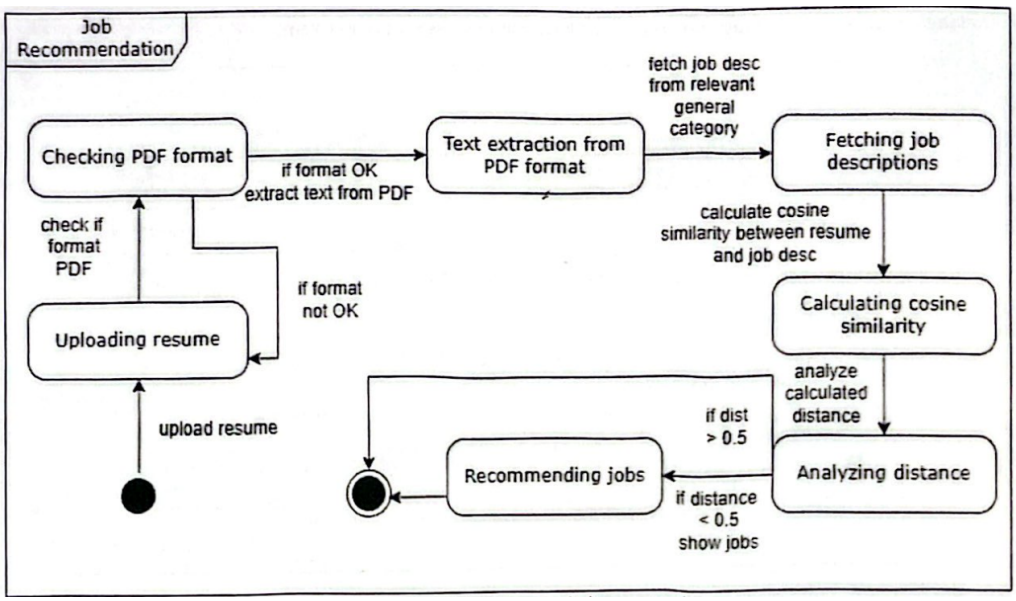

Figure 4.7: State Transition Diagram (Sample)

4.6.1 Level 1 Data Flow Diagram

The Level 1 DFD decomposes the platform into its major sub-processes: the Authentication and Verification process, the Property Listing and Search process, the Booking and Payment process, the Roommate Matching process, and the Administration process. Each process exchanges data with the relevant data stores, namely the Users, Properties, Bookings, Payments, Agreements, and Roommate Profiles collections, while external entities (tenant, landlord, administrator) provide inputs and receive outputs.

4.6.2 Level 2 Data Flow Diagram

The Level 2 DFD further decomposes the Booking and Payment process. It details the sub-processes of validating booking availability, creating the booking record, notifying the landlord, recording payment details, confirming payment, and triggering agreement generation, showing the data flowing into and out of the Bookings, Payments, and Agreements data stores at each step.

4.7 Data Design

This section describes how data is structured, stored, and managed. Domavi uses MongoDB, a document-oriented NoSQL database, accessed through the Mongoose object data modelling library. The schema is organised into discrete collections that mirror the domain entities. Documents reference one another by ObjectId to model relationships, and indexes are defined on frequently queried fields, including a 2dsphere geospatial index on property location and compound indexes on status fields, to ensure efficient queries. Security mechanisms include password hashing, role-based access control, and restricted access to verification documents.

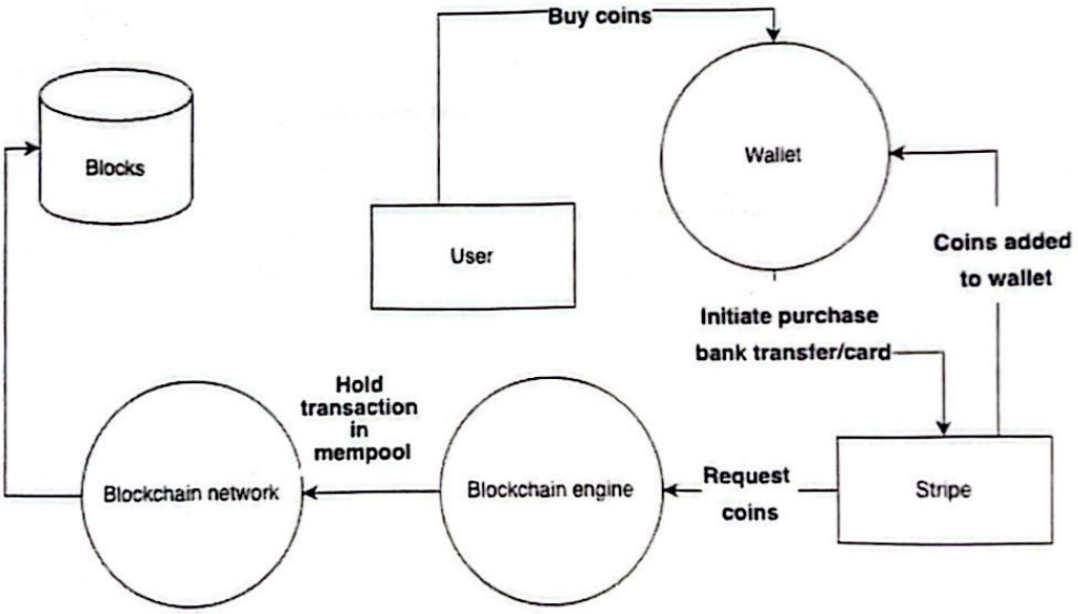

Figure 4.8: Level 1 Data Flow Diagram (Sample)

4.7.1 Entity Relationship Diagram (ERD)

The ERD represents the logical structure of the Domavi database. The principal entities are User, Property, RoommateProfile, Booking, Agreement, Payment, Conversation, Message, Document, and LandlordBankAccount. A User (landlord) owns many Properties (1:N); a User (tenant) makes many Bookings (1:N); a Property has many Bookings (1:N); a Booking produces one Agreement (1:1) and one or more Payments (1:N); a Conversation contains many Messages (1:N) and links two Users; and a User has one RoommateProfile (1:1). *(Replace the sample figure with the finalised Domavi ERD before submission.)*

4.7.2 Database Architecture

Domavi uses a client-server database architecture in which the Express application server is the sole client of the MongoDB database. Property images and verification documents are stored in cloud media storage, with the database retaining only their secure references. This separation keeps the database lean and allows media to be served efficiently. The design supports cloud deployment and can be scaled through MongoDB replication and sharding if required.

4.8 Database Schema Diagram

The schema diagram represents the physical structure of the collections, including field names, types, and the reference relationships between collections. The design supports data integrity through schema validation in Mongoose, efficient querying through indexing, scalability through a document model that avoids costly joins, and security through controlled access to sensitive fields. *(Replace the sample figure with the finalised Domavi schema diagram.)*

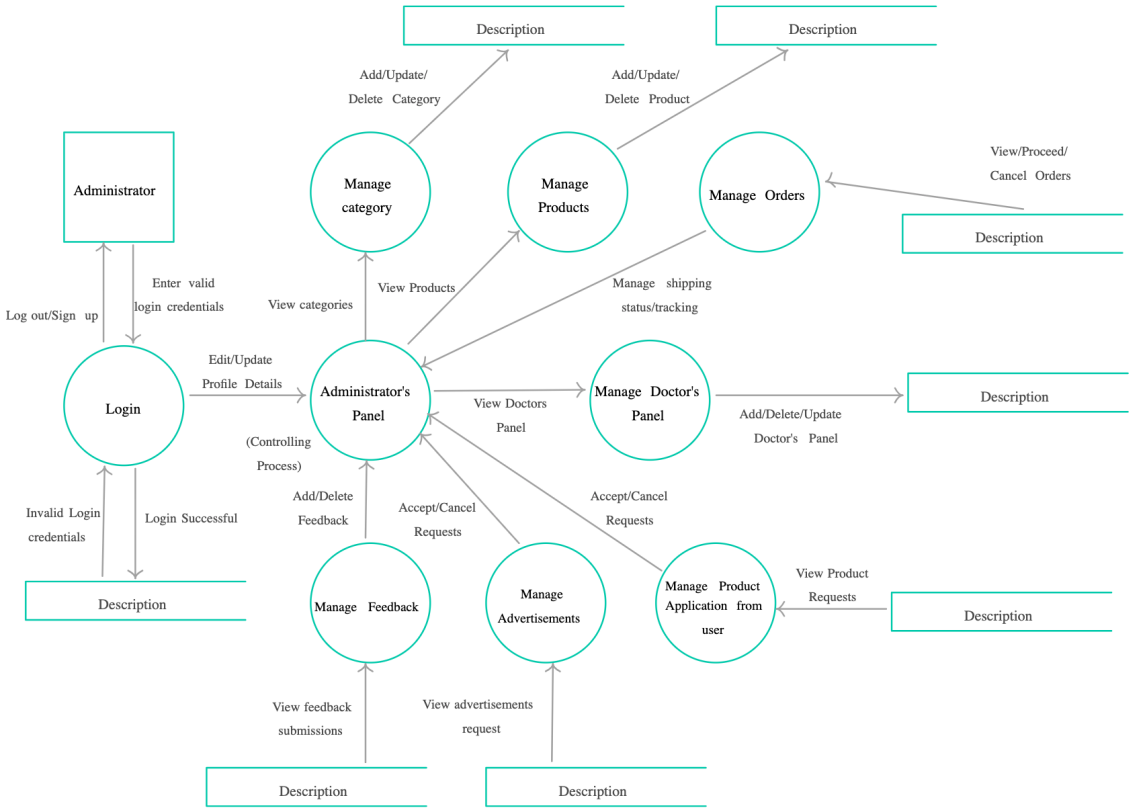

Figure 4.9: Level 2 Data Flow Diagram (Sample)

4.8.1 Data Dictionary

The data dictionary defines the structure of the principal data elements of the User collection as a representative example.

4.9 Database Collections

This section describes the logical database design of Domavi. As a NoSQL system, the database is organised into collections, each modelling a domain entity with primary identifiers, references, and major attributes. The design ensures data consistency, integrity, security, and scalability.

4.9.1 User Collection

The User collection stores authentication credentials and profile information for all roles and supports role-based access control and secure authentication.

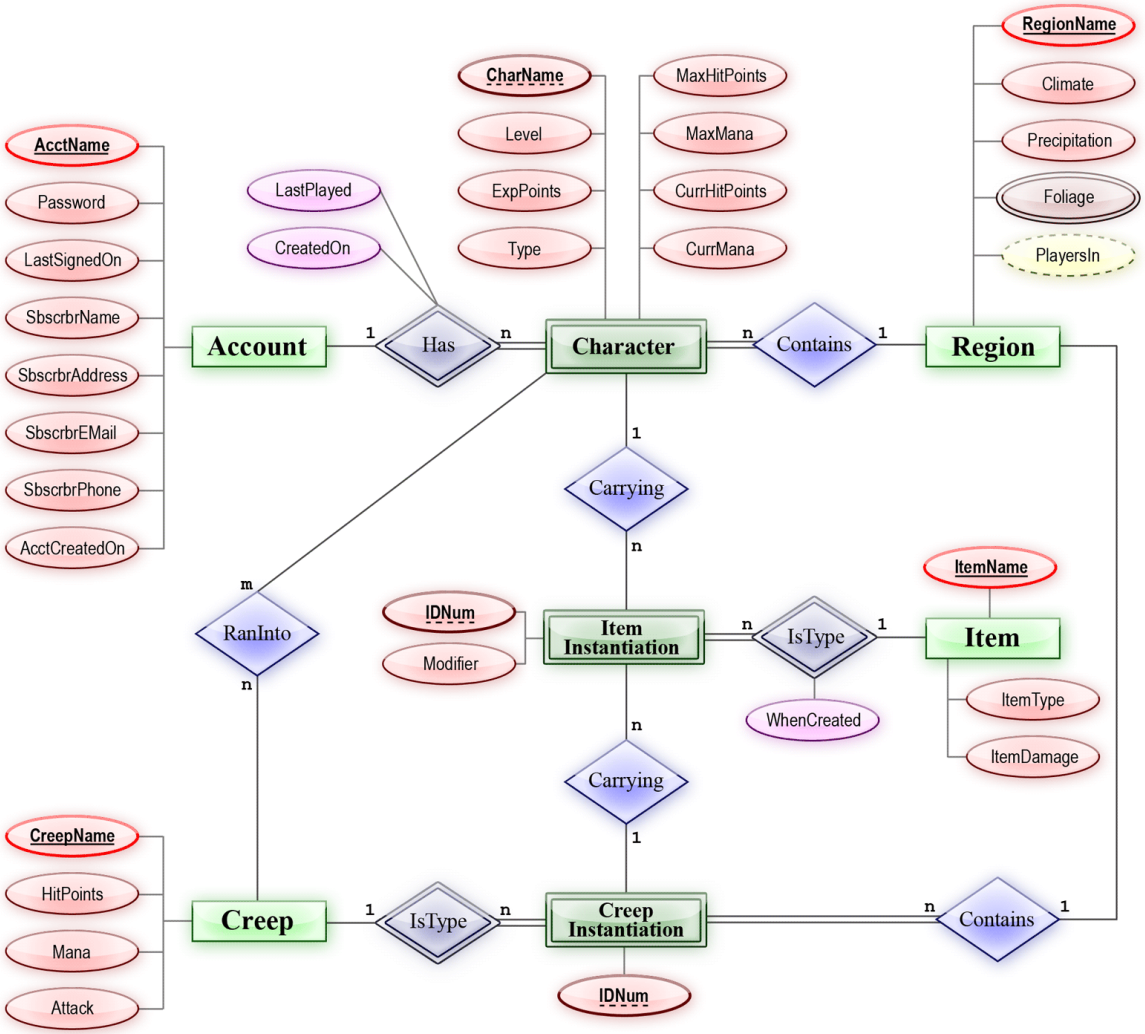

Figure 4.10: Entity Relationship Diagram (Sample)

4.9.2 Property Collection

The Property collection stores micro-rental listings created by landlords, including location as a GeoJSON point to support geospatial search.

4.9.3 Booking, Payment, and Agreement Collections

The **Booking** collection records each rental request, referencing the tenant, landlord, and property, and tracking the booking status and move-in date. The **Payment** collection records payment type, method, amount, bank details, and status (pending or confirmed), referencing the related booking, tenant, and landlord. The **Agreement** collection stores the auto-generated rental agreement reference and terms for a confirmed booking. The **RoommateProfile** collection stores each user's lifestyle, preferences, budget, location, and schedule data along with cached compatibility scores. The **Conversation** and **Message** collections store in-app messaging, while **Document** and **LandlordBankAccount** store verification documents and landlord payout details respec-

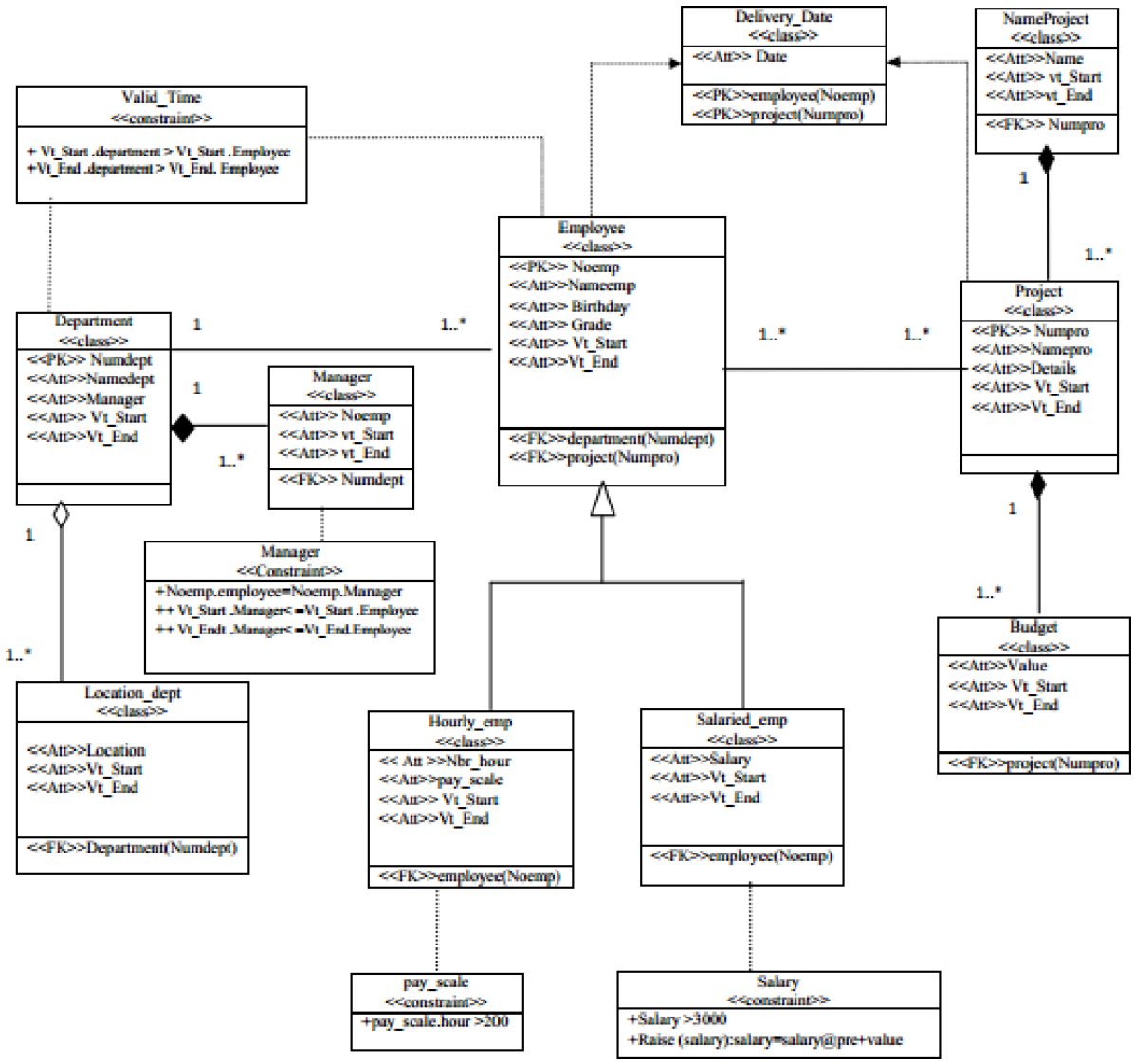

Figure 4.11: Database Schema Diagram (Sample)

tively.

4.9.4 Relationships Between Collections

- **One-to-One (1:1):** Each user has one roommate profile; each confirmed booking has one agreement.
- **One-to-Many (1:N):** A landlord owns many properties; a tenant makes many bookings; a property has many bookings; a conversation has many messages.
- **Many-to-Many (M:N):** Tenants interact with many properties through bookings and conversations, and properties are viewed by many tenants, resolved through the Booking and Conversation collections.

Table 4.1: Data Dictionary – User Collection

Field Name	**Type**	**Description**
_id	ObjectId	Unique identifier for each user
name	String	Full name of the user
email	String	Unique registered email address
passwordHash	String	Securely hashed password
role	String	User role: tenant, landlord, or admin
status	String	Account status: pending, active, or suspended
isVerified	Boolean	Indicates whether identity verification is complete
createdAt	Date	Account creation timestamp

Table 4.2: User Collection Structure

Field Name	**Data Type**	**Constraint**	**Description**
_id	ObjectId	Primary Key	Unique identifier
name	String	Not Null	User's full name
email	String	Unique, Not Null	Login email address
passwordHash	String	Not Null	Hashed password
role	String	Not Null	tenant / landlord / admin
status	String	Default pending	Account status
createdAt	Date	Default now	Creation timestamp

4.9.5 Security and Data Integrity

- Password hashing using a secure algorithm (bcrypt) with salting.
- Role-based access control (RBAC) enforced through authentication middleware.
- Schema-level validation and constraints enforced by Mongoose.
- Secure communication over HTTPS and authenticated API requests via JWT.
- Restricted access to verification documents and regular database backups.

4.9.6 Scalability Considerations

- Indexing on frequently queried attributes, including a geospatial index for location-based search.
- A document data model that avoids expensive joins and supports horizontal scaling.
- Offloading of media to cloud storage to keep the database lightweight.
- Support for MongoDB replication and sharding for future growth.

Table 4.3: Property Collection Structure

Field Name	**Data Type**	**Constraint**	**Description**
_id	ObjectId	Primary Key	Unique identifier
landlord	ObjectId	Foreign Key (User)	Owner of the listing
title	String	Not Null	Listing title
type	String	Not Null	room / bed space / hostel
price	Number	Not Null	Rent amount
location	GeoJSON Point	2dsphere index	Geographic coordinates
status	String	Default pending_verification	Listing status

4.10 Chapter Summary

This chapter presented the comprehensive software design of the Domavi platform. It justified an object-oriented, layered MVC design methodology and an Agile process model, and described the system through a context diagram and a three-layer client-server architecture. Detailed UML models, including activity, class, sequence, and state transition diagrams, were developed to describe the system's behaviour and structure, and data flow diagrams illustrated internal data movement. The chapter further elaborated the database design through the entity relationship model, schema diagram, data dictionary, and collection specifications, addressing relationships, security, integrity, and scalability. Overall, this chapter establishes a clear technical blueprint that guides the implementation phase discussed in the next chapter.

Chapter 5
Implementation

This chapter describes the practical implementation of the Domavi platform. It explains how the design presented in Chapter 4 was translated into working modules, algorithms, user interfaces, and integrated services using the MERN technology stack.

5.1 Development Environment

The platform was developed using a modern JavaScript and TypeScript toolchain. The development environment is summarised below.

- **IDE / Editor:** Visual Studio Code with ESLint and Prettier for linting and formatting.
- **Languages:** TypeScript across both the frontend and backend, with HTML and CSS in the presentation layer.
- **Frontend Framework:** React with the Vite build tool, Tailwind CSS for styling, and Zustand for state management.
- **Backend Framework:** Node.js with Express.js, organised into routes, controllers, services, and middleware.
- **Database:** MongoDB with the Mongoose object data modelling library.
- **Version Control:** Git with a GitHub repository for collaboration between the two team members.
- **Operating System:** Cross-platform development on Windows and macOS.
- **Hardware:** Standard development laptops with 8–16 GB RAM; no specialised GPU is required.

5.2 Core Module Implementation

Each major module of Domavi was implemented as a cohesive set of routes, a controller, and a service class on the backend, with corresponding pages and components on the frontend.

5.2.1 Module 1: Authentication and Verification

This module implements registration, login, email verification, and password reset. The authentication service hashes passwords with bcrypt, issues JSON Web Tokens on successful login, and exposes middleware that validates tokens and enforces role-based access control. The verification service manages the upload of CNIC and ownership documents to cloud storage and records verification requests for administrative review.

5.2.2 Module 2: Property Listing and Search

The property service handles the creation, update, retrieval, and removal of listings. Each property stores its location as a GeoJSON point, and a 2dsphere index enables commute-based proximity queries. The search functionality combines geospatial filtering with attribute filters for price, room type, and gender preference, returning only active and verified listings to tenants.

5.2.3 Module 3: Booking, Payment, and Agreement

The booking service manages the lifecycle of a booking from pending request to acceptance, confirmation, and completion. The payment service records payment details and bank information and updates the payment status upon administrative confirmation. Once a booking is confirmed, the agreement service generates a digital rental agreement as a PDF and stores its reference against the booking.

5.2.4 Module 4: Roommate Matching

The roommate matching module is implemented in a dedicated matcher service that computes a weighted compatibility score between roommate profiles. Profiles capture lifestyle, preferences, budget, location, and schedule attributes. The service computes a score per category, applies category weights, aggregates them into an overall score, caches results to avoid recomputation, and returns a ranked list of top matches.

5.2.5 Module 5: Messaging and Administration

The chat service manages conversations and messages between tenants and landlords. The administration module provides endpoints for reviewing verification requests, managing user accounts and status, overseeing payments, and retrieving analytics, all protected by an admin-only access guard.

5.3 Algorithm Implementation

The most significant algorithm in Domavi is the roommate compatibility matching algorithm. It quantifies how well two users would live together by comparing their lifestyle profiles across five weighted categories and producing an overall compatibility score on a scale of 0 to 100.

5.3.1 Algorithm: Weighted Roommate Compatibility Scoring

Input: Two roommate profiles, P_1 and P_2
Output: Overall compatibility score $S \in [0, 100]$

The algorithm evaluates five categories with the following default weights: lifestyle (0.25), preferences (0.25), budget (0.20), location (0.15), and schedule (0.15). For each category it computes a normalised similarity score between the two profiles, multiplies it by the category weight, and sums the weighted scores to obtain the overall result. To find matches for a given user, the algorithm computes this score against all candidate profiles and returns the highest-scoring candidates.

Step-by-Step Working:

1. Retrieve the requesting user's roommate profile.
2. For each candidate profile:
   - Compute the lifestyle similarity score.
   - Compute the preferences similarity score.
   - Compute the budget similarity score.
   - Compute the location similarity score.
   - Compute the schedule similarity score.
   - Multiply each category score by its weight and sum to obtain the overall score.
   - Cache the overall score for reuse.
3. Sort candidates in descending order of overall score.
4. Return the top N candidates as matches.

Pseudo Code:

```
Algorithm RoommateMatch(P_user, Candidates, Weights):
1. matches = []
2. For each C in Candidates:
      lifestyle  = similarity(P_user.lifestyle,   C.lifestyle)
      prefs      = similarity(P_user.preferences, C.preferences)
      budget     = similarity(P_user.budget,      C.budget)
      location   = similarity(P_user.location,    C.location)
      schedule   = similarity(P_user.schedule,    C.schedule)
      score = lifestyle*0.25 + prefs*0.25 + budget*0.20
              + location*0.15 + schedule*0.15
      matches.add({ candidate: C, score: round(score) })
3. Sort matches by score descending
4. Return top N matches
```

Explanation: The weighted approach allows the most behaviourally significant factors, lifestyle and preferences, to influence the result most strongly, while still accounting for budget, location, and schedule. Caching previously computed scores reduces repeated computation when the same pair is evaluated again. The algorithm integrates with the backend and is triggered when a tenant opens the roommate matches page.

5.4 External APIs / SDKs

Domavi integrates several third-party services to deliver its features.

Table 5.1: External APIs and Services Used

API / Service	Purpose	Used In Module
Google Maps API	Geocoding of addresses, proximity-based search, and map display with location pins.	Property Search
Cloud Media Storage	Secure storage and delivery of property images and verification documents.	Property, Verification
SMTP Email Service	Email verification and event notifications.	Authentication, Notifications

5.5 User Interface Implementation

This section presents the major user interface screens of Domavi. Each screen is described in terms of its purpose, components, interaction flow, backend integration, and validation. Students should include screenshots of each major screen in the final report.

5.5.1 UI Design Approach

The Domavi interface follows a clean, minimalistic design built with Tailwind CSS and a component library organised using the atomic design methodology, in which small atoms (buttons, inputs, badges) compose into molecules (property cards, search bars, filter chips) and organisms (complete sections). A consistent colour scheme, typography, and spacing scale are defined as design tokens to ensure visual consistency. The layout follows a mobile-first, responsive strategy so that the application adapts smoothly from mobile to desktop, and accessibility considerations such as readable font sizes and clear focus states are applied throughout.

5.5.2 Screen-wise Implementation

5.5.2.1 Screen 1: Authentication Screen

Purpose: Allows tenants and landlords to register and log in.
Components: Email field, password field, role selector, login and sign-up buttons, and a forgot-password link.
Functional Flow: On submission, the client validates the inputs and calls the authentication API; on success a token is stored and the user is redirected to their role-specific dashboard.
Backend Integration: Calls the authentication endpoints; the server validates credentials and returns a JWT.
Validation: Required-field, email-format, and password-strength validation with clear inline error messages.

5.5.2.2 Screen 2: Search and Listings Screen

Purpose: Enables tenants to discover accommodation near a chosen location.
Components: Location input, filter controls (price, type, gender preference), a results list of property cards, and an interactive map.
Functional Flow: Entering a location triggers a geospatial search; results update in both the

list and the map.
Backend Integration: Calls the property search endpoint with location and filter parameters.
Validation: Graceful empty-state handling when no listings match.

5.5.2.3 Screen 3: Add Property Screen

Purpose: Allows verified landlords to create a new listing.
Components: Form fields for title, description, type, price, amenities, location picker, and image upload.
Functional Flow: The landlord completes the form and submits; the listing is created with pending-verification status.
Backend Integration: Calls the property creation endpoint and uploads images to cloud storage.
Validation: Required fields, valid price, and at least one image.

5.5.2.4 Screen 4: Roommate Matches Screen

Purpose: Displays a ranked list of compatible roommates.
Components: Compatibility cards showing the candidate and their match score.
Functional Flow: On load, the system computes and displays the top matches for the user's profile.
Backend Integration: Calls the matching endpoint.
Security Considerations: Only accessible to authenticated tenants with a completed profile.

5.5.2.5 Screen 5: Admin Dashboard Screen

Purpose: Provides administrators with oversight of the platform.
Components: Navigation, summary data cards, verification queue, user management table, payment oversight, and analytics charts.
Functional Flow: Data loads dynamically from administrative endpoints.
Backend Integration: Calls admin-only endpoints.
Security Considerations: Protected by role-based access control restricting access to administrators.

5.5.3 Navigation Flow Diagram

A navigation flow diagram should be included to show how the major screens, the authentication, search, property detail, booking, payments, messages, roommate matches, and admin screens, are connected and how users move between them according to their role. *(Insert the finalised navigation flow diagram in the final report.)*

5.6 Chapter Summary

This chapter described the implementation of the Domavi platform using the MERN stack and TypeScript. It outlined the development environment, detailed the implementation of each core module, and presented the weighted roommate compatibility algorithm together with its pseudo code. The external APIs for mapping, media storage, and email were documented, and the major

user interface screens were described in terms of their purpose, components, flow, integration, and validation. The next chapter presents the testing and evaluation of the implemented system.

Chapter 6
Testing and Evaluation

This chapter presents the testing strategies and evaluation procedures used to verify the correctness, reliability, and performance of the Domavi platform. Testing ensures that all functional and non-functional requirements defined in earlier chapters are satisfied. As Domavi is a web application, the evaluation focuses on unit, integration, system, performance, security, and user acceptance testing.

6.1 Testing Strategy

Domavi was tested using a combination of automated and manual approaches applied throughout the Agile development cycle. Automated unit tests were written for backend services and controllers using the Jest testing framework, allowing individual functions such as authentication, booking creation, payment recording, and compatibility scoring to be verified in isolation. Integration testing validated the interaction between the React frontend, the Express REST API, and the MongoDB database, with API endpoints exercised to confirm correct request handling, authorisation, and data persistence. System testing was performed manually by walking through complete user journeys for each role, while performance and security testing assessed responsiveness and the robustness of authentication and access control. This layered strategy, moving from the smallest units up to the full system and finally to real users, ensured that defects were caught early and that the platform met its specified requirements.

6.2 Unit Testing

Unit testing verified individual functions and modules independently. Each major backend service was tested with valid and invalid inputs, and the expected output and pass/fail status were recorded.

6.2.1 UT-01: User Registration and Login

Table 6.1: Unit Test Cases – Authentication

Test ID	Input	Expected Output	Result
UT-01	Valid registration details	Account created with pending status	Pass
UT-02	Missing or invalid email	Validation error displayed	Pass
UT-03	Correct login credentials	JWT issued and access granted	Pass
UT-04	Incorrect password	Access denied with error	Pass

Table 6.2: Unit Test Cases – Roommate Matching

Test ID	Input	Expected Output	Result
UT-05	Two identical profiles	Compatibility score near 100	Pass
UT-06	Two opposite profiles	Low compatibility score	Pass
UT-07	List of candidates	Ranked matches sorted by score	Pass

6.2.2 UT-02: Roommate Compatibility Scoring

6.3 Integration Testing

Integration testing ensured that the platform's modules interact correctly, focusing on communication between the frontend and backend, the correct handling of authenticated API requests, and reliable database persistence. Key cross-module flows such as creating a listing and then searching for it, and submitting a booking and then recording a payment, were validated end to end.

Table 6.3: Integration Testing Summary

Test ID	Modules Integrated	Expected Result	Result
IT-01	Frontend + Backend API	Data retrieved and rendered correctly	Pass
IT-02	Backend + Database	Records stored and retrieved correctly	Pass
IT-03	Property + Maps API	Geospatial search returns nearby listings	Pass
IT-04	Booking + Payment + Agreement	Confirmed booking generates agreement	Pass

6.4 System Testing

System testing validated the complete application workflow for each user role. The tenant journey, registering, searching, viewing a property, messaging a landlord, submitting a booking, making a payment, and receiving an agreement, was tested end to end, as were the landlord journey of verification, listing, and booking management, and the administrator journey of verification review, user management, and analytics. Edge cases such as searches with no results, rejected bookings, unconfirmed payments, and access attempts to unauthorised resources were tested to confirm correct error handling and graceful degradation.

6.5 Performance Testing

Performance testing measured the responsiveness of key operations under normal conditions. Page loads and search results were observed to render within the target of approximately three seconds,

and indexed geospatial queries returned results quickly. The indicative metrics below summarise observed performance during testing.

Table 6.4: Performance Metrics

Metric	Description	Value
Response Time	Average API response time	≈ 220 ms
Search Latency	Geospatial search response time	< 1 s
Dashboard Load	Time to render main dashboard	< 3 s

6.6 Security Testing

Security testing focused on authentication, authorisation, and data protection. Authentication was verified by confirming that protected endpoints reject requests without a valid JWT. Authorisation was tested by confirming that role-based access control prevents tenants from accessing landlord or administrator functions and vice versa. Password storage was checked to ensure that only salted bcrypt hashes are persisted, never plaintext passwords. Input handling was tested against malformed and malicious inputs, and the use of parameterised Mongoose queries mitigates injection risks. Communication over HTTPS and restricted access to verification documents were also verified.

6.7 User Acceptance Testing

User acceptance testing was conducted with a small group of representative users, primarily students, who performed realistic tasks such as searching for a room, viewing roommate matches, and submitting a booking request. Feedback was generally positive, confirming that the platform addresses a real need, and the suggestions gathered were used to refine the interface and performance.

Table 6.5: User Feedback Summary

Feedback	Action Taken
Navigation could be simpler	Navigation and menus simplified
Search felt slightly slow on first load	Queries and indexes optimised
Wanted clearer match explanations	Compatibility scores made more visible

6.8 Testing Summary

A comprehensive set of test cases was executed across unit, integration, system, performance, security, and acceptance testing. The unit and integration test cases passed, confirming that individual functions and module interactions behave as specified. System testing validated complete user journeys for all roles, performance testing showed responsive behaviour within target

thresholds, and security testing confirmed that authentication, authorisation, and data protection mechanisms function correctly. User acceptance testing validated that the platform meets real user needs. On the basis of these results, the Domavi platform is considered to satisfy its functional and non-functional requirements and to be ready for demonstration and deployment.

Chapter 7
Conclusion and Future Work

7.1 Conclusion

This project set out to address a clear and pressing gap in the housing market: the difficulty that students and young professionals in Pakistan face when searching for affordable shared accommodation and compatible roommates. The existing rental ecosystem, dominated by informal brokers, unverified listings, and a complete absence of structured roommate matching, imposes high cost, fraud risk, wasted time, and avoidable interpersonal conflict on a large and growing population. No existing platform, whether a general classified site, a property portal, an international roommate service, or an informal social media group, adequately serves the micro-rental segment.

In response, this project designed and implemented **Domavi**, a web-based micro-rental and roommate matching platform that unifies several capabilities within a single ecosystem. The platform provides a dedicated marketplace for single rooms, bed spaces, and hostels; an intelligent roommate matching engine that ranks potential roommates using a weighted, multi-factor compatibility score; a multi-layer trust and verification system covering email, CNIC, and property ownership; commute-based geospatial search powered by the Google Maps API; and end-to-end automation through secure messaging, structured bookings, payment recording, and auto-generated digital rental agreements. The system was built on the MERN stack with TypeScript, following a layered, role-based architecture supporting tenants, landlords, and administrators.

The key achievements of the project include a fully functional platform spanning ten interconnected data collections and a complete REST API, a working weighted compatibility algorithm that replaces the roommate "gamble" with data-driven matching, and a verification workflow that meaningfully addresses the trust problem. The system was validated through unit, integration, system, performance, security, and user acceptance testing, all of which confirmed that it satisfies its functional and non-functional requirements and performs responsively under normal conditions.

Overall, Domavi makes a tangible contribution by bringing structure, transparency, and safety to a previously informal process. It demonstrates that a focused, verification-driven, compatibility-aware platform can deliver real social value to an underserved market while remaining technically sound and commercially viable, fulfilling the objectives established at the outset of this Final Year Project.

7.2 Future Work

While Domavi delivers a complete and working platform, several enhancements are envisioned for future development. The most immediate is the development of native Android and iOS mobile applications to complement the responsive web client and reach users on the devices they use most. A second priority is the integration of a live payment gateway, including local options such as JazzCash and Easypaisa, to enable real-time, in-app transactions rather than recorded bank transfers.

The roommate matching engine offers significant scope for advancement. Future versions could incorporate machine learning to learn from successful matches and user feedback, refining the compatibility weights and similarity functions over time to improve match quality. The verification process could be partially automated through document scanning and identity-verification services, reducing the administrative burden and accelerating onboarding. Additional features such as in-app review and rating systems for tenants and landlords, real-time push notifications, multilingual support, and an analytics dashboard for landlords would further enrich the platform. Finally, scalability enhancements such as database sharding, caching layers, and load balancing would prepare the platform for nationwide growth, while continued security hardening would maintain user trust as the user base expands.

7.3 Limitations

The current version of Domavi has a number of limitations. As a web-only platform, it requires an active internet connection and does not offer native mobile applications or offline functionality. The verification of CNIC and ownership documents relies on manual administrator review, which, while ensuring trust, limits the speed of fully automated onboarding. Payment handling in the initial release is based on recording and confirming bank-transfer details rather than processing live transactions through an integrated gateway. The roommate matching algorithm uses fixed category weights rather than weights learned from data, and the platform's initial coverage is focused on selected cities and university areas rather than nationwide. Finally, as an academic project developed by a two-member team within a fixed timeline, certain advanced features were intentionally scoped for future work, as described above.

Appendices

Appendix A - Turnitin Similarity Report

This appendix contains the official Turnitin Similarity Report generated for the final submitted version of this dissertation.

Plagiarism Report Screenshot:

Appendix B - AI Writing Detection Report

This appendix contains the AI Writing Detection Report generated by Turnitin (if applicable), verifying compliance with institutional academic integrity guidelines regarding AI-assisted content.

AI Detection Report Screenshot:

Appendix C – Source Code

This section provides details about the project source code repository.

Guidelines for Students:

- Upload your complete project source code to a version control platform such as GitHub, GitLab, or Bitbucket.
- Ensure the repository includes:
  - All source files
  - Database scripts (if applicable)
  - Model training and testing code (for ML projects)
  - Frontend and backend code (for web/mobile projects)
  - README file with setup instructions
- The repository must be properly structured with meaningful folder names.
- Remove unnecessary files (temporary files, cache files, datasets if restricted).
- If the repository is private, make the repository public before final submission.
- Paste the final repository link below.

Repository Link:

`PasteYourGitHubRepositoryLinkHere`

Additional Notes:

- Mention branch name if different from `main`.
- Mention if any third-party APIs require API keys.
- Mention hardware requirements (if ML/GPU-based project).